Optimising a City Bus Timetable

A from-scratch textbook for the CTU fleet-allocation project

Only high-school mathematics is assumed.
All demand numbers in the project are **synthetic and illustrative**.

Study guide written from the project's code, data and outputs

October 2026

Contents

How to use this book **v**

I Mathematics from the ground up **1**

1 Variables, parameters and units **2**

1.1 What a variable is . . . 2
1.2 Subscripts: one name for many boxes . . . 3
1.3 Units . . . 3
1.4 The most important conversion: headway and frequency . . . 3
1.5 Unit discipline: multiplying units . . . 4

2 Functions and graphs **6**

2.1 What a function is . . . 6
2.2 Reading a graph . . . 6
2.3 Linear and nonlinear functions . . . 7
2.4 Functions of many inputs and the sum sign Σ . . . 8

3 Inequalities and constraints **9**

3.1 Inequalities . . . 9
3.2 One variable: the number line . . . 9
3.3 Two variables: a half-plane . . . 10
3.4 Constraints . . . 10

4 Feasible regions **12**

4.1 Many constraints at once . . . 12
4.2 The two-route teaching example . . . 12
4.3 Binding and slack constraints . . . 13
4.4 When there is no feasible region . . . 14

5 Objective functions and optimisation **16**

5.1 What we are trying to achieve . . . 16
5.2 Minimising and maximising are the same job . . . 16
5.3 Trade-offs: when two objectives disagree . . . 17

6 Linear programming **18**
6.1 What makes a problem a linear program . . . 18
6.2 The graphical method, step by step . . . 18
6.3 Why the answer is at a corner . . . 19
6.4 What can go wrong . . . 19
6.5 The general form . . . 20

7 How solvers work (intuition only) **21**
7.1 What a solver is . . . 21
7.2 The simplex method: walking along corners . . . 21
7.3 What the solver reports . . . 22

8 Integer and binary variables **24**
8.1 Why fractions are a problem . . . 24
8.2 Rounding is not enough . . . 24
8.3 Binary variables: yes/no switches . . . 25
8.4 Rounding up with an integer variable . . . 26

9 Mixed-integer linear programming (MILP) **27**
9.1 Definition . . . 27
9.2 LP relaxation . . . 27
9.3 Branch and bound, step by step . . . 27
9.4 The optimality gap . . . 28

10 Modelling tricks that keep the model linear **30**
10.1 Trick 1: a menu of choices instead of a curve . . . 30
10.2 Trick 2: “max(0, something)” with an extra variable . . . 31
10.3 Trick 3: “at least $f\tau$ buses” with an integer . . . 31
10.4 Trick 4: “close to today” as simple bounds . . . 31
10.5 Trick 5: a soft rule as a penalty . . . 31
10.6 Trick 6: tie-breaks and locks . . . 32

II The bus world **33**

11 How a bus network runs **34**
11.1 Routes, terminals and loops . . . 34
11.2 Cycle time . . . 34
11.3 Little’s law: how many buses a frequency needs . . . 36
11.4 Calibration: which reading of d_r is right? . . . 36
11.5 Fleet, spares and the usable fleet . . . 37
11.6 Capacity, load factor and overflow . . . 37
11.7 Vehicle-kilometres . . . 38

11.8 Time periods and partial service 38

12 Waiting time **39**

12.1 Why the average wait is half the headway 39
12.2 Demand-weighted waiting 40
12.3 Irregular buses wait longer: the $1 + \mathrm{CV}^2$ factor 40

13 Demand and its uncertainty **42**

13.1 Why the demand is synthetic 42
13.2 The recipe 42
13.3 Uncertainty as a triangle: fuzzy numbers 43
13.4 The α-cut: turning a triangle into one planning number 43
13.5 How to read a heatmap 45

III Many goals at once **47**

14 Many objectives: domination and the Pareto front **48**

14.1 Dominance 48
14.2 The Pareto front 48
14.3 Ideal and nadir 49
14.4 Normalisation 50
14.5 The knee point 50
14.6 Weighted sums and their weakness 50

15 Goal programming **51**

15.1 Goals instead of “as good as possible” 51
15.2 The project’s four goals 51
15.3 Method 1: preemptive (priority) goal programming 52
15.4 Method 2: weighted goal programming 52
15.5 Method 3: Chebyshev (min-max) goal programming 53
15.6 The final “tidy” solve 53
15.7 Results in the project ($\alpha = 0.5$, relaxed $f_{\max}$) 53

16 Computing a Pareto front: exact method and genetic algorithm **55**

16.1 The ε-constraint method 55
16.2 AUGMECON: two refinements 56
16.3 NSGA-II: a genetic algorithm 56
16.4 Measuring the quality of a front 57
16.5 Why use both methods? 58

17 Simulation, statistics and sensitivity **60**

17.1 Random variables and distributions 60

17.2 Seeds: making randomness repeatable . . . 60
17.3 Monte Carlo simulation . . . 61
17.4 Summarising many results . . . 61
17.5 How to read a box plot . . . 62
17.6 The Gini coefficient . . . 62
17.7 Sensitivity analysis . . . 63

IV Our project, assembled 64

18 The complete model 65
18.1 Sets (what we loop over) . . . 65
18.2 Parameters (fixed numbers) . . . 65
18.3 Decision variables . . . 65
18.4 Constraints . . . 66
18.5 Objectives . . . 66
18.6 The different “runs” . . . 67

19 How to read every figure in the project 69
19.1 Block diagram of the method . . . 69
19.2 Calibration: buses needed vs fleet . . . 70
19.3 Demand heatmap . . . 71
19.4 Fuzzy demand and α-cuts . . . 72
19.5 Goal-programming scenarios . . . 72
19.6 Pareto front, 2-D projections . . . 73
19.7 Pareto front in 3-D . . . 74
19.8 Simulated trade-offs . . . 74
19.9 Wait vs vehicle-km . . . 75
19.10 Overflow risk box plots . . . 76
19.11 Tornado: sensitivity of the waiting gain . . . 76
19.12 Price of robustness (α) . . . 77
19.13 Fleet reduction . . . 78
19.14 Out-of-sample test . . . 78
19.15 Most crowded routes . . . 79

20 Results, limitations and what the project does not do 80
20.1 The headline, in numbers . . . 80
20.2 Five things to say about the results . . . 80
20.3 What the project does NOT do (known limitations) . . . 81
20.4 Things that look strange but are correct . . . 81
20.5 Where each number comes from . . . 81

Glossary of terms and symbols 83

How to use this book

What the project does, in one sentence

The project uses a computer to decide how often buses should run on each of Chandigarh's 68 CTU routes, in the morning peak, midday and evening peak, so that passengers wait less and fewer people are left without room, without using more buses than the city has.

That is the whole idea. Everything else in this book explains the pieces that make that sentence precise.

The pipeline in plain words

The project runs in six stages (the file `run_all.py` runs them in order):

1. **Data audit.** Read the CTU timetable spreadsheet: 68 routes, 389 buses, 1662 km of route, 649 stops. Check that every number adds up.
2. **Periods and demand.** Split the day into AM peak (07–10 h), Midday (10–16 h) and PM peak (16–19 h). Invent (*synthesise*) how many passengers each route has, because no real ridership data exists. Describe the uncertainty with a "triangle" of low, likely and high values.
3. **Optimisation model (MILP).** Write the timetable question as equations and inequalities and let a solver find the best answer.
4. **Goal programming.** Give the solver targets ("4% less waiting", "no long headways") and priorities, and see which targets can be met.
5. **Pareto front.** Map every "best possible compromise" between waiting, kilometres driven and overcrowding, with two different methods that check each other.
6. **Simulation.** Test the chosen timetables on 2000 random "days" of demand and on changed assumptions.

How each chapter is built

Every concept comes with the same boxes:

Definition: the box you are reading about

A plain-English definition. Read this first.

Intuition

An everyday analogy.

Worked example: name

Numbers, solved one step at a time.

Common confusion

The mistake most beginners make, and why it is wrong.

How this appears in our project

Where exactly this idea is used in the code and the results.

Check your understanding

Short questions. The answers are right below each question; cover them with your hand first.

A promise about honesty

This book never invents a result. Every number about the project comes from the files in `outputs/` (for example `outputs/stage3_report.md`, `outputs/gp_report.md`, `outputs/pareto_report.md`, `outputs/simulation_findings.md`) or from the settings in `src/params.py`. Teaching examples with small made-up numbers are clearly called "teaching example". And one sentence you should say out loud in every presentation: **the demand is synthetic, so the results show how the method behaves, not what CTU will achieve.**

Reading order

Part I (Chapters 1–10) is pure maths. Part II (11–13) is the bus world. Part III (14–17) is about handling many goals and uncertainty. Part IV (18–20) puts the real project together and explains every figure. If you are short of time before a presentation, read Chapters 11, 12, 18, 19 and 20 first, then come back for the rest.

Part I

Mathematics from the ground up

Chapter 1

Variables, parameters and units

1.1 What a variable is

Definition: variable

A **variable** is a named box that holds a number we do not know yet, or a number we are allowed to choose. We give it a short name, usually a letter such as x or h.

Intuition

Think of a recipe card that says "add x spoons of sugar". The letter x is a blank you fill in. Different cooks can fill in different numbers. The recipe (the rules) stays the same.

In this project there are two very different kinds of named numbers. Keeping them apart is the single most useful habit you can learn.

Definition: decision variable and parameter

A **decision variable** is a number the computer is allowed to choose (for example, how many buses run on route 31). A **parameter** is a number that is fixed before the computer starts (for example, the length of route 31, or the bus capacity of 60 passengers).

Worked example: sorting the numbers of a bus route

Route 31 in the CTU timetable is 37 km long and has 3 buses. We want to decide how often a bus leaves.

1. "Route length is 37 km": the city built the route; we cannot change it. **Parameter.**
2. "Bus capacity is 60 passengers": a fixed assumption. **Parameter.**
3. "A bus leaves every h minutes": this is what we want to choose. **Decision variable.**
4. "Number of buses on the route, x": we may move buses between routes. **Decision variable.**

Common confusion

"Variable" in everyday English means "something that changes". In optimisation, a parameter can also change (we can rerun with a different bus capacity), but *within one run* it is fixed. The test is: *who sets the number? If you set it before the run, it is a parameter. If the computer sets it during the run, it is a decision variable.*

1.2 Subscripts: one name for many boxes

The city has 68 routes. Writing $x_1, x_2, \ldots, x_{68}$ is easier than inventing 68 different letters.

Definition: subscript, index

A **subscript** is the small label below a letter: in x_r the letter r says *which* route. The label is called an **index**. Two indices are common in this project: r for the route and t for the time period. So x_{rt} means "buses on route r in period t".

Worked example: reading double subscripts

Periods are $t \in \{\text{AM}, \text{Midday}, \text{PM}\}$. The symbol $\in$ means "is one of".

- $x_{31,\text{AM}} = 4$ reads "route 31 uses 4 buses in the AM peak".
- $x_{31,\text{Midday}} = 3$ reads "route 31 uses 3 buses at midday".

These are two separate boxes. Changing one does not change the other.

1.3 Units

Definition: unit

A **unit** says what a number counts: minutes, kilometres, buses, passengers. A number without its unit is only half the information.

The project fixes these units everywhere (file `src/params.py`):

Quantity	Symbol	Unit	Example
headway (gap between buses)	h	minutes [min]	15
frequency (buses per hour)	f	departures per hour [dep/h]	4
distance	d	kilometres [km]	37
speed	v	kilometres per hour [km/h]	22
time for one round trip	τ ("tau")	hours [h]	1.5
demand	D	passengers per hour [pax/h]	110
bus capacity	C	passengers per bus [pax/bus]	60

"pax" is the transport industry's short word for passengers.

1.4 The most important conversion: headway and frequency

Definition: headway and frequency

The **headway** h is the number of minutes between one bus and the next on a route. The **frequency** f is how many buses leave per hour. An hour has 60 minutes, so

$$f = \frac{60}{h} \qquad \text{and} \qquad h = \frac{60}{f}.$$

Worked example: converting both ways

1. Headway 15 min. Frequency $f = 60/15 = 4$ buses per hour.
2. Headway 20 min. $f = 60/20 = 3$ dep/h.
3. Headway 80 min (route 31). $f = 60/80 = 0.75$ dep/h. Less than one bus per hour.
4. Frequency 6 dep/h. Headway $h = 60/6 = 10$ min.

Common confusion

Short headway means high frequency. A "15–30 min" timetable entry has a *shortest* headway of 15 and a *longest* of 30. The shortest headway gives the *highest* frequency: $f_{\max} = 60/15 = 4$. The longest gives the lowest: $f_{\min} = 60/30 = 2$. Students often mix these up. The code checks this rule (`src/data.py`, function `parse_headway_range`).

Common confusion

The average of frequencies is not the frequency of the average headway. For "15–30 min" the project's middle frequency is $f_{\text{mid}} = (2 + 4)/2 = 3$ dep/h. But the middle headway, 22.5 min, gives $60/22.5 = 2.67$ dep/h. Both are "middles". The project uses $f_{\text{mid}} = (f_{\min} + f_{\max})/2$, the same as the spreadsheet (decision D7 in `docs/DECISIONS.md`).

1.5 Unit discipline: multiplying units

Units multiply and cancel like numbers. This is how you can check a formula without understanding it fully.

Worked example: buses needed

A bus takes $\tau = 1.5$ h to do a full round trip. We want $f = 4$ departures per hour. How many buses?

$$f \times \tau = 4 \frac{\text{dep}}{\text{h}} \times 1.5 \text{ h} = 6 \text{ dep} = 6 \text{ buses}.$$

The "h" on top and bottom cancel. What is left is a count of departures in one round-trip time, which equals the number of buses that must be on the road at once. Chapter 11 explains why.

How this appears in our project

- `src/params.py` lists every unit at the top and holds the only two conversion functions, `headway_min_to_freq_per_h` and `freq_per_h_to_headway_min`.
- The decision variables are: y_{rtk} (which headway is chosen), x_{rt} (buses) and o_{rt} (overflow passengers). They are introduced in Chapters 8–10 and assembled in Chapter 18.
- Parameters include $C = 60$ pax/bus, speed $v = 22$ km/h, layover 10%, fleet $B = 389$ buses, usable fleet $B_{\text{eff}} = 357$, and the demand numbers.

Check your understanding

1. A route has a headway of 12 min. What is its frequency?

   Answer: $60/12 = 5$ departures per hour.

2. Is "bus capacity $C = 60$" a decision variable or a parameter in this project?

   Answer: A parameter: it is fixed before the solver runs.

3. What does $x_{40,\mathrm{PM}}$ mean?

   Answer: The number of buses on route 40 during the PM peak.

4. A timetable says "20–40 min". What are $f_{\max}$ and $f_{\min}$?

   Answer: $f_{\max} = 60/20 = 3$ dep/h (from the shortest headway), $f_{\min} = 60/40 = 1.5$ dep/h.

5. Check the units: D [pax/h] $\times$ H [h] gives what?

   Answer: Passengers. For example 100 pax/h for 3 h is 300 passengers.

Chapter 2

Functions and graphs

2.1 What a function is

Definition: function

A **function** is a rule that takes an input number and gives back exactly one output number. We write output $= g(\text{input})$, for example $W(h) = h/2$: put in a headway h, get back the average wait W.

Intuition

A vending machine: you press button h, one specific snack comes out. Pressing the same button twice always gives the same snack. A function never gives two different answers for the same input.

Worked example: the waiting-time function

If buses come exactly every h minutes and you arrive at a random moment, your average wait is half the headway (Chapter 12 proves this). So $W(h) = h/2$.

1. $W(10) = 10/2 = 5$ min.
2. $W(30) = 30/2 = 15$ min.
3. $W(80) = 80/2 = 40$ min. (Route 31 at its published 80-minute headway.)

2.2 Reading a graph

A graph draws a function. The **horizontal axis** (the x-axis) shows the input. The **vertical axis** (the y-axis) shows the output. Each point (a, b) says "input a gives output b".

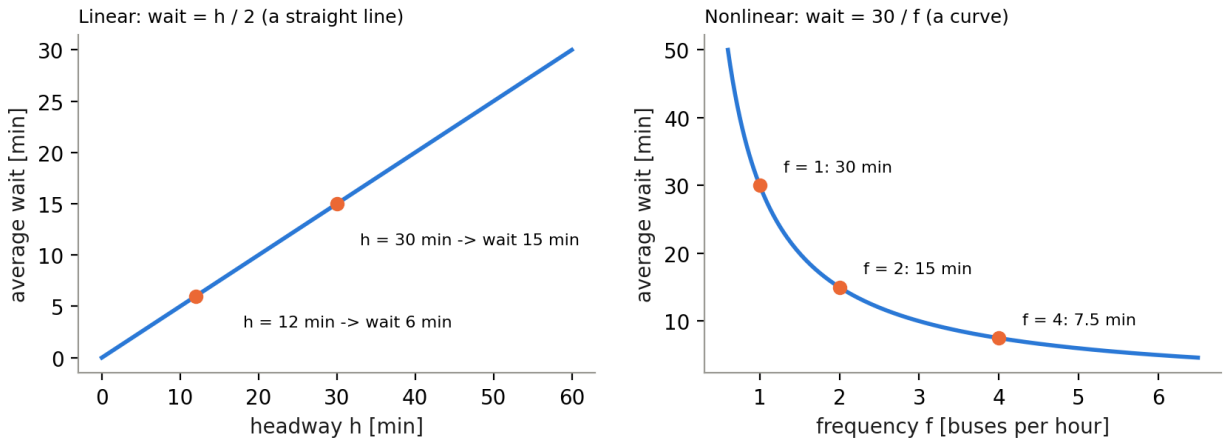


Figure 2.1: Left: wait as a function of headway is a straight line. Right: wait as a function of frequency is a curve.

How to read: a line graph

1. Read the axis labels and units first. Left panel: across is headway in minutes, up is wait in minutes.
2. Pick a point on the line, go straight down to read the input, straight left to read the output.
3. Ask: does the line go up or down as I move right? Left panel: up (longer headway, longer wait). Right panel: down (more buses, shorter wait).

2.3 Linear and nonlinear functions

Definition: linear function

A function is **linear** if its graph is a straight line. In symbols: $g(x) = a\,x + b$ for fixed numbers a and b. With several inputs: $g(x_1, x_2) = a_1 x_1 + a_2 x_2 + b$. Every input is only multiplied by a fixed number and then added. No input is squared, divided into, or multiplied by another input.

Definition: slope

The **slope** a says how much the output grows when the input grows by 1. For $W = h/2$ the slope is $1/2$: one more minute of headway adds half a minute of waiting.

Worked example: linear or not?

1. $W = h/2$: linear (slope $\frac{1}{2}$, no constant).
2. km $= 74 \times f \times 3$: linear in f (74 and 3 are fixed numbers).
3. $W = 30/f$: **not linear**. f sits in the denominator. Check: $f = 1$ gives 30, $f = 2$ gives 15, $f = 3$ gives 10. The drops are 15, then 5. A straight line would drop by the same amount each time.
4. $x_1 \times x_2$: not linear (two inputs multiplied together).

Common confusion

$W = h/2$ and $W = 30/f$ describe *the same physical fact* (because $h = 60/f$, so $h/2 = 30/f$). One is linear and one is not. Whether a formula is linear depends on *which quantity you treat as the input*. The project uses this: it chooses the headway, so the waiting formula can be made linear (Chapter 10).

2.4 Functions of many inputs and the sum sign Σ

The project adds up quantities over 68 routes and 3 periods. The sum sign saves writing.

Definition: summation

$\sum_r x_r$ means "add up x_r for every route r". $\sum_r \sum_t x_{rt}$ means "add up over every route and every period".

Worked example: a sum by hand

Three routes have buses $x_A = 4$, $x_B = 5$, $x_C = 3$. Then $\sum_r x_r = 4 + 5 + 3 = 12$ buses. If each route's waiting is "demand times half the headway", with demands 300, 200, 150 pax/h and headways 10, 15, 20 min:

$$\sum_r D_r \frac{h_r}{2} = 300 \cdot 5 + 200 \cdot 7.5 + 150 \cdot 10 = 1500 + 1500 + 1500 = 4500 \text{ pax} \cdot \text{min per hour}.$$

How this appears in our project

- Waiting $Z_1 = \sum_{r,t} M_{rt} \cdot h_{rt}/2$ (demand-weighted waiting) is exactly the sum in the last example, for 191 route-periods (`src/model.py`, `build_core`).
- The project deliberately avoids $30/f$ because solvers for linear problems cannot handle curves.

Check your understanding

1. What is $W(24)$ for $W(h) = h/2$?

   Answer: 12 minutes.

2. Is $Z = 3x + 5y$ linear? Is $Z = 3x^2$?

   Answer: The first is linear. The second is not (x is squared).

3. Using $W = 30/f$, how much does waiting fall when frequency goes from 1 to 2? From 5 to 6?

   Answer: From 30 to 15 (15 min). From 6 to 5 (1 min). Extra buses help less and less.

4. Compute $\sum_r x_r$ for $x = (2, 0, 7)$.

   Answer: 9.

Chapter 3

Inequalities and constraints

3.1 Inequalities

Definition: inequality

An **inequality** compares two quantities without saying they are equal. The four signs are $<$ (less than), $\leq$ (less than or equal to), $>$ (greater than), $\geq$ (greater than or equal to).

Worked example: true or false

1. $3 \leq 5$: true.
2. $5 \leq 5$: true ("or equal").
3. $5 < 5$: false (strictly less is not allowed).
4. $357 \geq 385$: false. (Keep these two numbers in mind: 357 is the number of buses the model may use, 385 is the number the published timetable uses.)

3.2 One variable: the number line

The set of numbers that make an inequality true can be drawn on a number line.

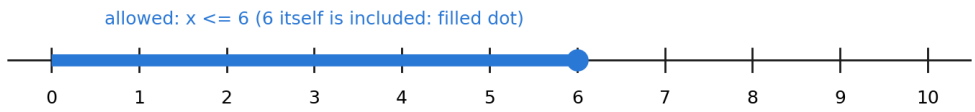


Figure 3.1: The numbers $0 \leq x \leq 6$ (with x counting buses, so x cannot be negative).

Worked example: solving a one-variable inequality

A route needs at least $f \cdot \tau$ buses. With $f = 3$ dep/h and $\tau = 1.2$ h, the rule is $x \geq 3 \times 1.2 = 3.6$. Buses come in whole numbers, so $x \in \{4, 5, 6, \dots\}$. The smallest choice is 4.

3.3 Two variables: a half-plane

With two variables, an inequality like $x_A + x_B \leq 10$ splits the flat plane into two halves.

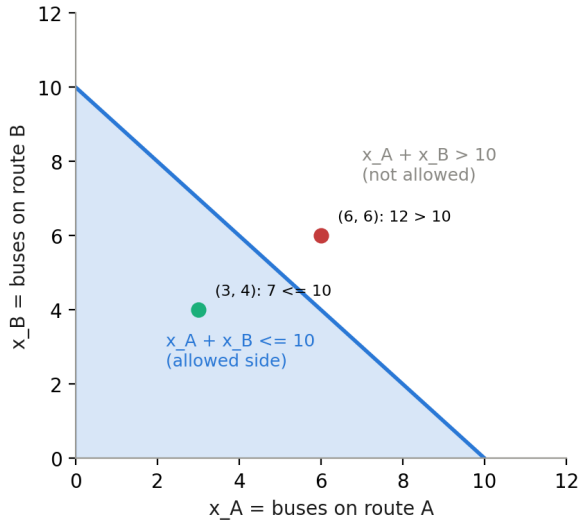


Figure 3.2: The line $x_A + x_B = 10$ is the border. The shaded side is allowed.

Worked example: which side is allowed?

To find the allowed side, test one easy point, usually $(0, 0)$:

1. Put $x_A = 0$, $x_B = 0$ into $x_A + x_B \leq 10$: we get $0 \leq 10$, true.
2. So the side containing $(0, 0)$ is allowed. Shade it.
3. Check $(3, 4)$: $7 \leq 10$, allowed. Check $(6, 6)$: $12 \leq 10$ is false, not allowed.

3.4 Constraints

Definition: constraint

A **constraint** is a rule that every acceptable answer must obey, written as an equation or an inequality in the decision variables. A **hard** constraint can never be broken. (Chapter 15 introduces *soft* goals that may be missed at a cost.)

Intuition

Packing a suitcase for a flight. "At most 23 kg" is a hard constraint: the airline refuses a 24 kg bag. "I'd like to bring 5 books" is a wish (a goal), not a constraint.

Worked example: turning words into constraints

Two routes A and B share a depot with 10 buses.

1. "The depot has 10 buses": $x_A + x_B \leq 10$.
2. "Route B's terminal can hold at most 7 buses": $x_B \leq 7$.
3. "You can't run negative buses": $x_A \geq 0$, $x_B \geq 0$. (These are called **non-negativity constraints** and are easy to forget.)

4. "Route A must have exactly as many buses as last year (4)": $x_A = 4$. An equation is also a constraint.

Common confusion

$\leq$ versus $<$. Optimisation software always uses $\leq$ and $\geq$, never strict $<$ or $>$. Why? With "$x < 10$" there is no largest allowed number (9.9, 9.99, 9.999 ...), so "the best" might not exist. With "$x \leq 10$" the best is simply 10.

Common confusion

Direction flips when you multiply by a negative number. $-x \leq -3$ means $x \geq 3$. When moving terms around in a constraint, multiplying both sides by -1 reverses the sign.

How this appears in our project

The model has six families of hard constraints (written C1–C6 in `docs/MODEL.md`). In words:

- C1: in each period, total buses used ≤ 357.
- C2: each route has enough buses to run its frequency, $x \geq f \cdot \tau$.
- C3: overflow passengers $\geq$ demand $-$ capacity.
- C4: the frequency stays inside the allowed range.
- C5: buses per route stay close to today's number; headways should stay under a cap.
- C6: a route that does not run in a period gets no buses in that period.

Chapter 18 gives every one with numbers.

Check your understanding

1. Is $(5, 6)$ allowed by $x_A + x_B \leq 10$?

   Answer: No: $5 + 6 = 11 > 10$.

2. Write "route A needs at least 2 more buses than route B" as a constraint.

   Answer: $x_A \geq x_B + 2$, or equivalently $x_A - x_B \geq 2$.

3. Rewrite $-2x \geq -8$ without minus signs.

   Answer: Divide by -2 and flip the sign: $x \leq 4$.

4. $f = 2$ dep/h and $\tau = 2.6$ h. What is the smallest whole number of buses?

   Answer: $2 \times 2.6 = 5.2$, so at least 6 buses.

Chapter 4

Feasible regions

4.1 Many constraints at once

Definition: feasible solution and feasible region

A **solution** is one choice of values for all decision variables. It is **feasible** if it obeys *every* constraint. The set of all feasible solutions is the **feasible region**.

Intuition

Choosing a restaurant with friends. One friend says "under 500 rupees", another "vegetarian options", a third "within 2 km". Each rule crosses some restaurants off. The restaurants left after all three rules are the feasible region. If none are left, the group has to relax a rule.

4.2 The two-route teaching example

We will use this teaching example in Chapters 4–9. It is small enough to draw, and has the same shape as the real problem.

Worked example: two routes sharing a depot

A depot runs two routes, A and B. Decide x_A and x_B, the number of buses on each.

- Fleet: 10 buses in total, so $x_A + x_B \leq 10$.
- Kilometres: a bus on A drives 30 km per hour, a bus on B 15 km per hour, and the depot's fuel budget allows 250 km per hour: $30x_A + 15x_B \leq 250$.
- Route B's terminal fits at most 7 buses: $x_B \leq 7$.
- No negative buses: $x_A \geq 0$, $x_B \geq 0$.

Steps to draw the feasible region:

1. Draw each border as a line. For $30x_A + 15x_B = 250$: when $x_A = 0$, $x_B = 250/15 = 16.7$; when $x_B = 0$, $x_A = 250/30 = 8.33$. Join these two points.
2. For each line, test $(0, 0)$ to find the allowed side. All three are satisfied by $(0, 0)$, so the allowed side is always the side towards the origin.
3. The feasible region is where all allowed sides overlap: the shaded shape in the figure.
4. Find the **corner points**, where two borders meet. For the fleet line and the km line,

solve $x_A + x_B = 10$ and $30x_A + 15x_B = 250$ together. From the first, $x_B = 10 - x_A$. Substitute: $30x_A + 15(10 - x_A) = 250$, so $15x_A = 100$, $x_A = 6.67$, $x_B = 3.33$.

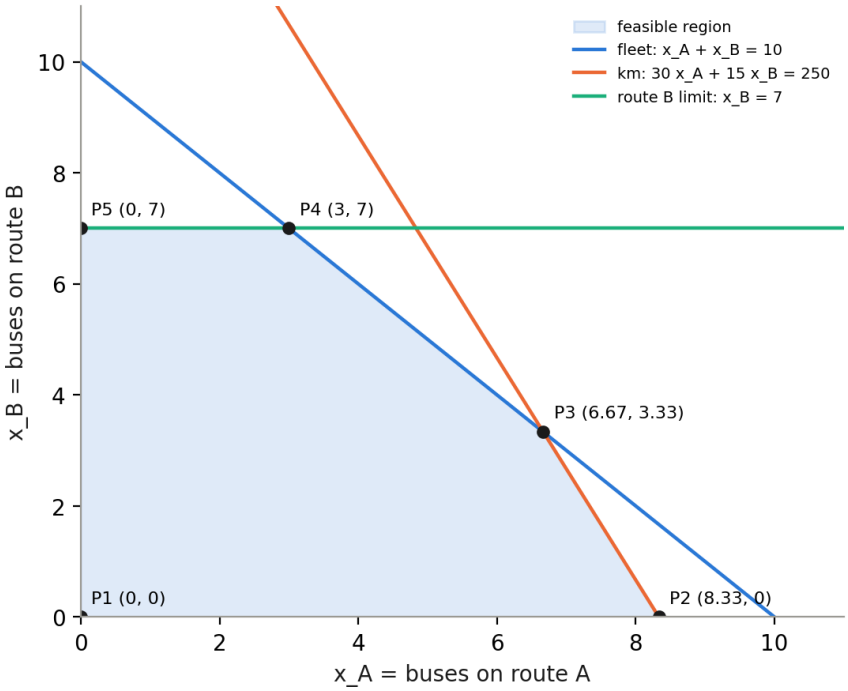


Figure 4.1: The feasible region of the teaching example and its five corner points P1–P5.

How to read: a feasible-region diagram

- Each coloured line is one constraint's border. Its label tells you which rule it is.
- The shaded polygon is everything allowed. A point outside it breaks at least one rule.
- The corners matter most: Chapter 6 shows the best answer of a linear problem is always at a corner.

4.3 Binding and slack constraints

Definition: binding, slack

At a given solution, a constraint is **binding** (or "tight") if it holds with equality: the solution sits exactly on its border. Otherwise it has **slack**: the gap between the two sides.

Worked example: binding at P3

At P3 $= (6.67, 3.33)$: fleet $6.67 + 3.33 = 10$, binding. Kilometres $30 \cdot 6.67 + 15 \cdot 3.33 = 250$, binding. Terminal $3.33 \leq 7$, slack $= 7 - 3.33 = 3.67$ buses.

4.4 When there is no feasible region

Definition: infeasible

A problem is **infeasible** if no solution obeys all the constraints at once. The feasible region is empty.

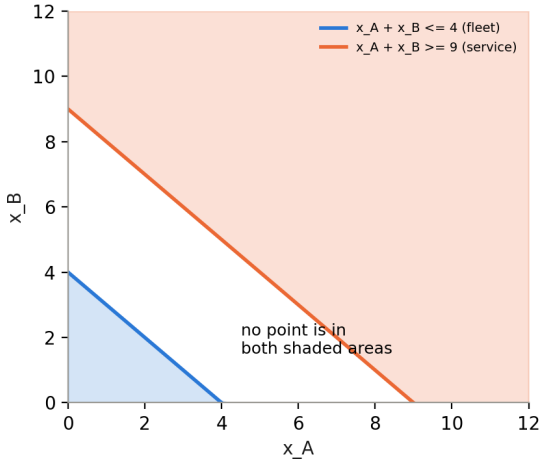


Figure 4.2: "At most 4 buses" and "at least 9 buses" cannot both be true.

Common confusion

"Infeasible" does not mean "the computer failed". It is a real finding: *the rules contradict each other*. The project reports infeasible settings honestly. For example, with a 10% spare ratio the usable fleet is 350 buses, but the minimum published frequencies already need 352 buses in the AM peak, so no timetable exists (`outputs/simulation_findings.md`).

How this appears in our project

- The real feasible region has 651 dimensions (one per variable in the published-bounds model), so it cannot be drawn. The idea is identical: the solver searches inside the region cut out by all constraints.
- `src/model.py`, function `presolve_report`, checks for infeasibility *before* solving and explains it in plain words, for example "AM: minimum buses 352 exceed B_eff = 350".
- Binding constraints tell the story of a result. In the project the fleet limit C1 is binding: the optimised schedules use all 357 buses.

Check your understanding

1. Is $(8, 1)$ feasible in the teaching example?

   Answer: Fleet $9 \leq 10$ yes; km $240 + 15 = 255 \leq 250$ no. Infeasible.

2. Which constraints are binding at P4 $= (3, 7)$?

   Answer: Fleet $(3 + 7 = 10)$ and terminal $(x_B = 7)$. Km: $90 + 105 = 195 < 250$, slack 55.

3. What does "the model is infeasible" tell a manager?

Answer: That the rules cannot all be satisfied together; one of them must be relaxed (for example, more buses or lower minimum frequencies).

Chapter 5

Objective functions and optimisation

5.1 What we are trying to achieve

Definition: objective function

The **objective function** is the single number we want to make as large as possible (**maximise**) or as small as possible (**minimise**). It is written in terms of the decision variables.

Definition: optimal solution

An **optimal solution** is a feasible solution with the best objective value. "Optimisation" means finding it. The best value itself is called the **optimal value** or **optimum**.

Intuition

Planning a road trip. The constraints are "arrive before 6 pm" and "spend at most 2000 rupees on fuel". The objective might be "see as many sights as possible". Change the objective to "spend as little as possible" and the best plan changes, even though the rules stay the same.

Worked example: objective of the teaching example

In the two-route example, suppose each bus on route A carries 60 passengers per hour and each bus on B carries 45. Objective: maximise passengers carried, $Z = 60x_A + 45x_B$.

1. At $(0, 0)$: $Z = 0$.
2. At $(3, 7)$: $Z = 180 + 315 = 495$.
3. At $(6, 4)$: $Z = 360 + 180 = 540$.

We want the feasible point with the largest Z. Chapter 6 finds it.

5.2 Minimising and maximising are the same job

Maximising Z gives the same best solution as minimising $-Z$. So software usually only knows "minimise". The project minimises everything: waiting, kilometres, overflow.

5.3 Trade-offs: when two objectives disagree

Worked example: waiting against kilometres

More frequent buses mean less waiting but more kilometres driven. Take one route with demand 100 pax/h, cycle length 20 km, running for 3 hours (teaching example).

headway h [min]	$f = 60/h$	waiting $100 \cdot h/2$ [pax · min/h]	km $= 20 \cdot f \cdot 3$
10	6	500	360
15	4	750	240
30	2	1500	120

No headway is best on both. This is a **trade-off**. Part III is all about handling it.

Common confusion

"Optimal" always means "optimal *for this objective, under these constraints, with these numbers*". Change the demand (here it is synthetic), and the optimal timetable can change. This is why the project runs a simulation and a sensitivity analysis (Chapter 17).

How this appears in our project

The project has three objectives, all minimised:

- Z_1: passenger waiting, in passenger-minutes per hour.
- Z_2: vehicle-kilometres per day (a measure of operating cost).
- Z_3: unserved passengers per day (people who do not fit on a bus).

Stage 3 minimises Z_3 alone (`src/stage3.py`). Stages 4 and 5 handle all three at once.

Check your understanding

1. Compute $Z = 60x_A + 45x_B$ at $(5, 5)$.

   Answer: $300 + 225 = 525$.

2. Why might a bus company not want to minimise waiting alone?

   Answer: Because the cheapest way to cut waiting is more frequent buses, which costs kilometres, fuel and drivers, and needs more buses than it has.

3. "Minimise Z" and "maximise $-Z$": same best solution?

   Answer: Yes.

Chapter 6

Linear programming

6.1 What makes a problem a linear program

Definition: linear program (LP)

A **linear program** is an optimisation problem where

1. the objective function is linear,
2. every constraint is a linear equation or a linear inequality ($\leq$, $\geq$, $=$), and
3. the decision variables may take any real value (fractions allowed) inside the constraints.

6.2 The graphical method, step by step

With two variables we can solve an LP by drawing. Here is the full recipe on the teaching example.

Worked example: solving the two-route LP by hand

Maximise $Z = 60x_A + 45x_B$ subject to $x_A + x_B \leq 10$, $30x_A + 15x_B \leq 250$, $x_B \leq 7$, $x_A, x_B \geq 0$.
Step 1. Draw the feasible region (done in Chapter 4).
Step 2. List the corner points. Each corner is where two borders cross.

Corner	Which two borders	How we solved it	(x_A, x_B)
P1	$x_A = 0$, $x_B = 0$	the origin	$(0, 0)$
P2	$x_B = 0$, km line	$30x_A = 250$	$(8.33, 0)$
P3	fleet line, km line	$15x_A = 100$ (Chapter 4)	$(6.67, 3.33)$
P4	fleet line, $x_B = 7$	$x_A = 10 - 7$	$(3, 7)$
P5	$x_A = 0$, $x_B = 7$		$(0, 7)$

Step 3. Evaluate the objective at every corner.

Corner	$Z = 60x_A + 45x_B$	
P1	0	
P2	$60 \cdot 8.33 = 500$	
P3	$60 \cdot 6.67 + 45 \cdot 3.33 = 400 + 150 = 550$	**largest**
P4	$180 + 315 = 495$	
P5	315	

Step 4. Pick the best corner. P3 with $Z = 550$. (Notice it asks for 6.67 buses. We return to this problem in Chapter 8.)

6.3 Why the answer is at a corner

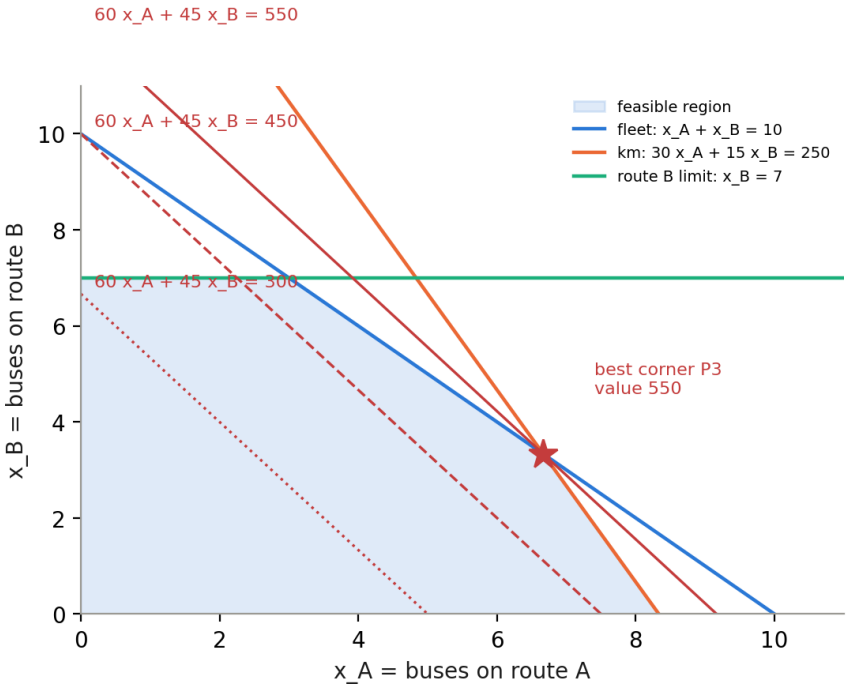


Figure 6.1: Red lines are *iso-objective lines*: every point on one line has the same Z. Pushing the line outwards raises Z. The last feasible point it touches is P3.

Intuition

Hold a ruler at the angle of the red lines and slide it across the shaded shape, away from the origin. The last point of the shape the ruler touches is the optimum. A straight ruler leaving a shape with straight edges always leaves at a corner (or along a whole edge, if the ruler is parallel to that edge, in which case a corner is still among the best).

Definition: corner-point theorem

If an LP has an optimal solution, at least one corner point of the feasible region is optimal. So you only ever need to check corners.

6.4 What can go wrong

1. **Infeasible**: no feasible region (Chapter 4).
2. **Unbounded**: the objective can improve forever. For example "maximise x" with only $x \geq 0$. Real problems are rarely unbounded; if a solver says so, a constraint has been forgotten.
3. **Several optima**: the best ruler position touches a whole edge. Any point on that edge is optimal. The project breaks such ties on purpose with tiny extra terms (Chapter 10).

6.5 The general form

With n variables we cannot draw, but the shape of the problem stays the same:

$$\min\ c_1x_1 + c_2x_2 + \cdots + c_nx_n \quad \text{subject to} \quad a_{i1}x_1 + \cdots + a_{in}x_n \le b_i \ \ (i = 1, \ldots, m), \quad x_j \ge 0.$$

Here the c_j, a_{ij} and b_i are parameters (fixed numbers) and the x_j are the decision variables.

Common confusion

"Programming" here does not mean computer programming. The word comes from the 1940s, when "programme" meant "plan" or "schedule" (as in a military supply programme). A linear program is a linear *plan*.

How this appears in our project

The real model is *almost* an LP: its objective and constraints are all linear. It is not a pure LP only because some variables must be whole numbers (buses) or 0/1 switches (which headway is used). That makes it a *mixed-integer* LP, the topic of Chapter 9.

Check your understanding

1. Change the objective to $Z = 60x_A + 90x_B$. Which corner is best?

   Answer: P1: 0; P2: 500; P3: $400 + 300 = 700$; P4: $180 + 630 = 810$; P5: 630. P4 $(3, 7)$ is best with 810. A new objective can move the answer to a different corner.

2. Why do we only check corners?

   Answer: The corner-point theorem: some corner is always optimal for an LP.

3. What would "unbounded" mean for a bus model?

   Answer: That some objective can be improved without limit, which signals a missing rule such as the fleet limit.

Chapter 7

How solvers work (intuition only)

7.1 What a solver is

Definition: solver

A **solver** is a program that takes an optimisation problem written as numbers (objective coefficients, constraint coefficients, right-hand sides) and returns the optimal solution, or a message saying there is none.

Definition: modelling library

A **modelling library** lets you write the problem in readable code and translates it into the numbers the solver needs. The project uses **PuLP** (version 4.0.1) as the modelling library and **HiGHS** as the solver (`src/model.py`, decision D28).

Intuition

PuLP is the waiter who writes down your order in the kitchen's code. HiGHS is the chef who cooks it. You never talk to the chef directly.

7.2 The simplex method: walking along corners

Chapter 6 showed the best answer is at a corner. With thousands of variables there are far too many corners to list. The **simplex method** (George Dantzig, 1947) avoids listing them:

1. Start at any corner of the feasible region.
2. Look at the neighbouring corners (one edge away). If one has a better objective, walk to it.
3. Repeat until no neighbour is better. Because the region is convex (it has no dents), a corner with no better neighbour is the global best.

Worked example: simplex on the teaching example

Maximise $60x_A + 45x_B$.

1. Start at P1 $(0, 0)$, $Z = 0$. Neighbours: P2 ($Z = 500$) and P5 ($Z = 315$). Walk to P2.
2. At P2, $Z = 500$. Neighbours: P1 (0) and P3 (550). Walk to P3.
3. At P3, $Z = 550$. Neighbours: P2 (500) and P4 (495). Neither is better. Stop: P3 is

optimal.

Three steps, without ever looking at P5's neighbour P4 from the other side.

Definition: convex

A region is **convex** if the straight line between any two of its points stays inside it. Feasible regions of LPs are always convex, because each constraint keeps a straight-edged half and the overlap of convex pieces is convex.

HiGHS also has an **interior-point method**, which travels through the middle of the region instead of along its edges. You do not need its details. For a presentation it is enough to say "HiGHS is a modern open-source LP and MILP solver developed at the University of Edinburgh; it uses simplex and interior-point methods for the linear parts and branch and bound (Chapter 9) for the integer parts".

7.3 What the solver reports

Status	Meaning
Optimal	the best solution was found and proven best (within a tolerance)
Infeasible	the constraints contradict each other
Unbounded	the objective can improve forever
Time limit	the solver stopped early; the answer may not be the best

Common confusion

"Optimal" comes with a tolerance. Computers work with rounded decimals. Solvers accept a constraint as satisfied if it is broken by less than about 10^{-6} to 10^{-9}. For integer problems, there is also an *optimality gap* (Chapter 9). The project tightens this gap to 10^{-6} in Stages 4 and 5 (`GAP_REL` in `src/goals.py`) so that "optimal" really means optimal.

How this appears in our project

- `src/model.py`, function `solve`: calls HiGHS, stops the run with an error unless the status is `Optimal`, and returns the schedule.
- Speed: the Stage 3 model (651 variables, 576 constraints) solves in 0.05 s (`outputs/stage3_report.md`). The whole 20 × 20 Pareto grid of Stage 5 (326 solves) takes about 60 s.
- PuLP 4 no longer ships the older CBC solver, which is why HiGHS is used (decision D28).

Check your understanding

1. What does the simplex method do at each step?

   Answer: It moves from the current corner to a better neighbouring corner, and stops when no neighbour is better.

2. Who solves the project's model: PuLP or HiGHS?

   Answer: HiGHS solves it; PuLP only builds it and passes it on.

3. What should you do if a solver reports “Time limit”?

   Answer: Treat the answer as possibly not optimal; give more time or simplify the model. The project stops with an error in that case.

Chapter 8

Integer and binary variables

8.1 Why fractions are a problem

The LP answer of the teaching example was $x_A = 6.67$, $x_B = 3.33$ buses. Two-thirds of a bus cannot drive.

Definition: integer variable

An **integer variable** may only take whole-number values: $0, 1, 2, 3, \ldots$ (and negatives, if allowed).

8.2 Rounding is not enough

Worked example: why you cannot just round

LP answer $(6.67, 3.33)$, value 550.

1. Round to the nearest: $(7, 3)$. Check km: $30 \cdot 7 + 15 \cdot 3 = 255 > 250$. **Infeasible.**
2. Round both down: $(6, 3)$. Feasible. Value $360 + 135 = 495$.
3. But $(6, 4)$ is feasible (fleet 10, km $180 + 60 = 240$) with value $360 + 180 = \mathbf{540}$, better than $(6, 3)$.

Rounding either breaks a rule or loses value. We need a method that searches whole numbers properly (Chapter 9).

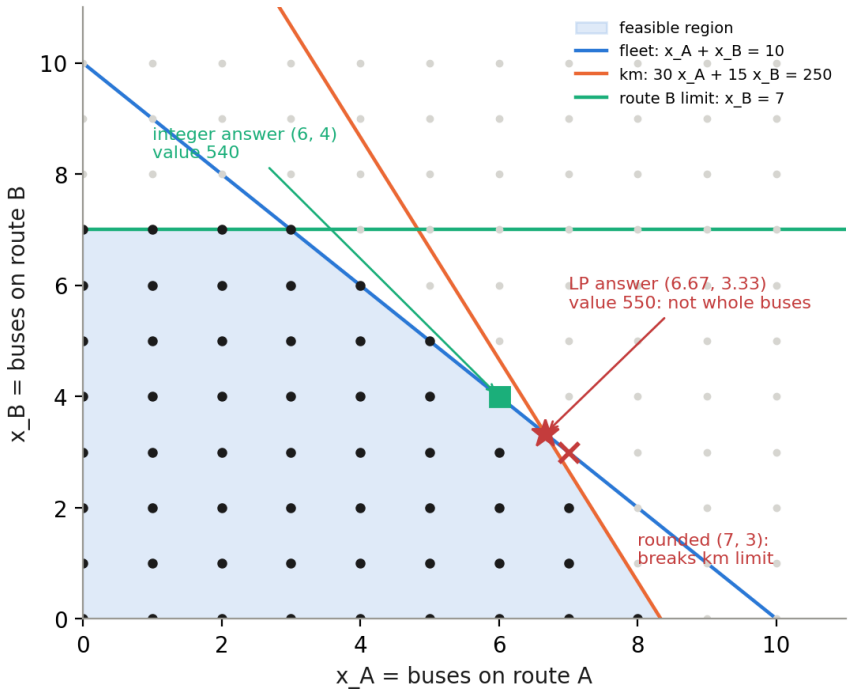


Figure 8.1: Black dots are the feasible whole-number points. The LP optimum (red star) is not one of them. The best dot is $(6, 4)$.

8.3 Binary variables: yes/no switches

Definition: binary variable

A **binary variable** is an integer variable that can only be 0 or 1. It works as a switch: 1 means "yes, do this", 0 means "no".

Intuition

A light switch. It is on or off; there is no "40% on". Binary variables let a linear model make yes/no decisions.

Worked example: choosing one headway from a menu

Route A may run every 10, 15 or 20 minutes. Make three switches $y_{10}, y_{15}, y_{20} \in \{0, 1\}$ with the rule

$$y_{10} + y_{15} + y_{20} = 1 \qquad \text{(exactly one switch is on)}.$$

Then the chosen headway and frequency are

$$h = 10y_{10} + 15y_{15} + 20y_{20}, \qquad f = 6y_{10} + 4y_{15} + 3y_{20}.$$

Check: choose 15 minutes, so $y_{15} = 1$ and the others are 0. Then $h = 15$ and $f = 4$. Correct. Both formulas are linear in the switches y.

8.4 Rounding up with an integer variable

Worked example: buses needed must be rounded up

Frequency 3 dep/h, cycle time 1.2 h. Buses needed $= 3 \times 1.2 = 3.6$. The constraint "$x \geq 3.6$ with x an integer" forces $x \geq 4$. The integer requirement does the rounding up automatically. The symbol for rounding up is $\lceil 3.6 \rceil = 4$ ("ceiling").

Common confusion

Binary does not mean "two values of my choice". A binary variable is exactly 0 or 1. To pick between 10 and 15 minutes you need switches multiplied by those numbers, as above.

Common confusion

Integers make problems much harder. An LP with a million variables can be solved quickly. A problem with a few hundred integer variables can, in the worst case, need a huge search. Our model is small enough that it is still fast.

How this appears in our project

- $y_{rtk} \in \{0, 1\}$: route r in period t uses headway k from its menu (`src/model.py`, `cat="Binary"`). Constraint `one_headway`: $\sum_k y_{rtk} = 1$.
- x_{rt}: integer number of buses (`cat="Integer"`).
- $o_{rt} \geq 0$: overflow passengers per hour, an ordinary continuous variable.

The menu is $K = \{10, 12, 15, 20, 25, 30, 40, 45, 60, 90\}$ minutes, plus each route's own published headways (decision D21, because some routes publish 35, 50, 55, 75 or 80 min).

Check your understanding

1. With menu $\{10, 20, 30\}$ and $y_{30} = 1$, what is f?

   Answer: $f = 60/30 = 2$ dep/h.

2. Can $y_{10} = y_{20} = 1$ if $\sum y = 1$?

   Answer: No; that sum would be 2.

3. Route 40 runs every 60 min ($f = 1$) and has $\tau = 3.08$ h. Minimum buses?

   Answer: $1 \times 3.08 = 3.08$, so 4 buses. It has only 3 today, which is why decision D25 widens the "close to today" rule to at least ± 1 bus.

4. Is rounding the LP answer a safe way to get an integer answer?

   Answer: No; it may break constraints or miss better whole-number points.

Chapter 9

Mixed-integer linear programming (MILP)

9.1 Definition

Definition: MILP

A **mixed-integer linear program** is a linear program in which *some* variables must be integers (or binary) and the others may be fractions. "Mixed" refers to that mixture.

How this appears in our project

The project's model is a MILP: y is binary, x is integer, o is continuous, and everything else is linear. Hence the phrase "corrected MILP" in the project title slides and in `docs/MODEL.md`.

9.2 LP relaxation

Definition: LP relaxation

The **LP relaxation** of a MILP is the same problem with the whole-number rules removed. It is an ordinary LP.

The relaxation can only be as good as or better than the MILP, because it allows more points. So for a maximisation problem, the relaxation's value is an **upper bound**: the true integer optimum cannot be higher.

9.3 Branch and bound, step by step

Worked example: branch and bound on the teaching example

Maximise $60x_A + 45x_B$ with the same constraints, now with x_A, x_B integers.
Node 0. Solve the LP relaxation: $(6.67, 3.33)$, value 550. Upper bound = 550. x_A is fractional.
Branch. Any integer x_A is either ≤ 6 or ≥ 7. Nothing in between is a whole number, so splitting this way loses no integer solution. Create two sub-problems.
Node 1: add $x_A \leq 6$. The best LP point is $(6, 4)$, value 540. Both are whole numbers: this

is a feasible integer solution. Record it as the **incumbent** (best so far): 540.
Node 2: add $x_A \geq 7$. With $x_A = 7$, the km limit leaves $(250 - 210)/15 = 2.67$ for x_B. Value $420 + 120 = 540$. Larger x_A only lowers the value. So the best this branch could ever reach is 540.
Bound (prune). Node 2's best possible is 540, which cannot *beat* the incumbent 540. There is no need to explore it further.
Finish. No open nodes remain. $(6, 4)$ with value 540 is proven optimal.

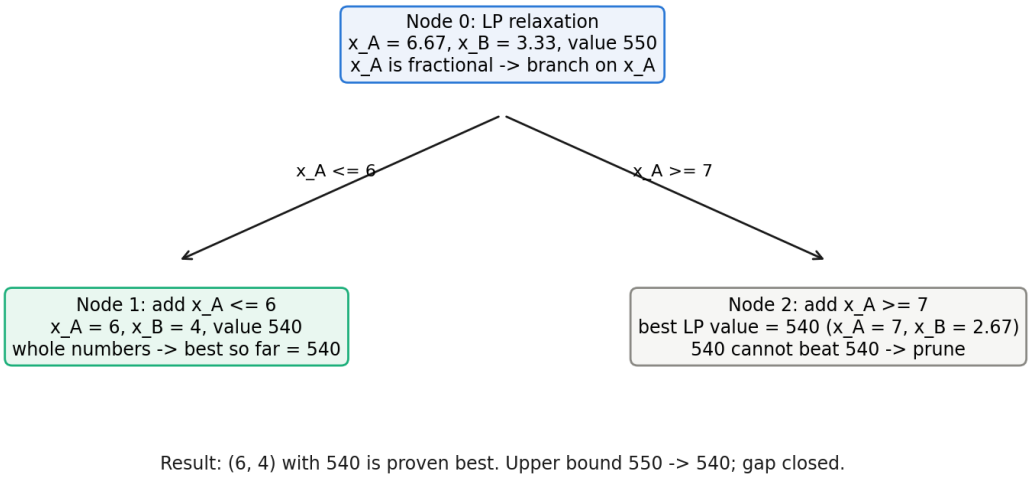


Figure 9.1: The branch-and-bound tree of the teaching example.

Intuition

Looking for the cheapest flight. You split the search by airline. If even the cheapest-possible fare of one airline is dearer than a ticket you already hold, you stop searching that airline. Branch and bound does exactly this with bounds from LP relaxations.

9.4 The optimality gap

Definition: MIP gap

During the search the solver knows the incumbent (a real solution) and a bound (no solution can be better than this). The **relative gap** is roughly $|\text{bound} - \text{incumbent}|/|\text{incumbent}|$. When the gap reaches the chosen tolerance, the solver stops and reports "Optimal".

Worked example: gap in numbers

At node 0 of the example, before any integer solution was found, there was no incumbent. After node 1: incumbent 540, bound 550, gap $= 10/540 = 1.9\%$. After pruning node 2: bound 540, gap 0.

Common confusion

A default gap of 10^{-4} (0.01%) sounds tiny, but on a number like $Z_1 \approx 180{,}000$ it allows an error of 18. When the project "locks" a goal's value and then optimises the next goal (Chapter 15), such errors could accumulate. That is why the project sets the gap to 10^{-6} (decision D30).

How this appears in our project

- Every solve in the project is a MILP solved by HiGHS's branch and bound.
- The published-bounds model of Stage 3 has 651 variables and 576 constraints. It is small: 191 active route-periods, each with a handful of headway switches, one bus count and one overflow variable.
- Because the solver proves optimality, the exact Pareto points of Stage 5 come with a certificate; the genetic algorithm (Chapter 16) does not have one.

Check your understanding

1. Why is branching on "$x_A \leq 6$ or $x_A \geq 7$" safe?

   Answer: Every integer value of x_A falls in one of the two branches; only fractions between 6 and 7 are thrown away.

2. In a maximisation, is the LP relaxation's value an upper or lower bound?

   Answer: An upper bound.

3. Incumbent 100, bound 101. Gap?

   Answer: About 1%.

Chapter 10

Modelling tricks that keep the model linear

Real rules are often not linear at first sight. This chapter shows the six tricks the project uses to write them linearly. Each one appears in `src/model.py` or `src/goals.py`.

10.1 Trick 1: a menu of choices instead of a curve

Waiting $= 30/f$ is a curve (Chapter 2). The project lets each route pick its headway from a menu with binary switches (Chapter 8). Then

$$\text{waiting} = M \sum_k \frac{h_k}{2} y_k, \qquad f = \sum_k \frac{60}{h_k} y_k.$$

Each $h_k/2$ and $60/h_k$ is a *fixed number*, computed before solving, so both formulas are linear in y.

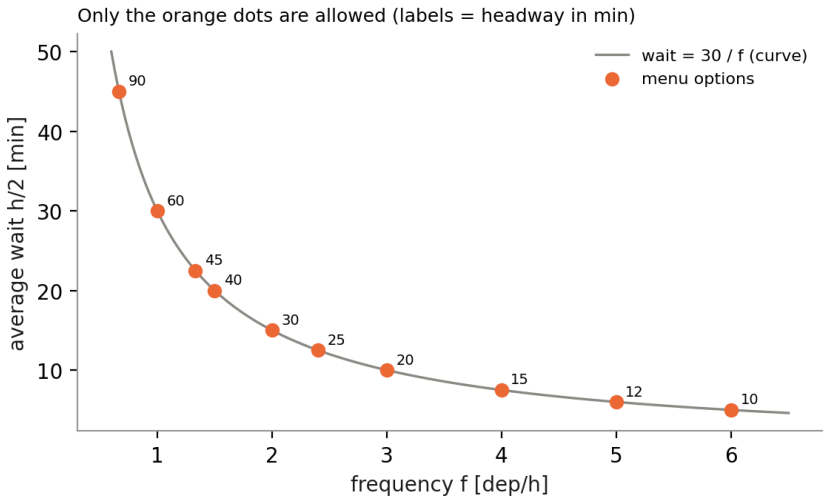


Figure 10.1: The curve is replaced by a finite set of allowed points (the project's menu K).

Worked example: route menu

Menu $\{10, 15, 20\}$, demand $M = 300$ pax/h. Waiting terms: $300 \cdot 5 = 1500$, $300 \cdot 7.5 = 2250$, $300 \cdot 10 = 3000$ pax $\cdot$ min/h. The model's waiting is $1500y_{10} + 2250y_{15} + 3000y_{20}$: linear.

10.2 Trick 2: "max(0, something)" with an extra variable

Overflow is "demand above capacity, or zero if everyone fits": $\max(0, D - Cf)$. The function max has a kink, so it is not linear. Replace it with a new variable o and two linear constraints:

$$o \geq D - Cf, \qquad o \geq 0.$$

Because the objective *minimises* overflow, the solver pushes o down until one of the two constraints is binding. So o ends up exactly equal to $\max(0, D - Cf)$.

Worked example: overflow by hand

$D = 129.5$ pax/h, $C = 60$, $f = 0.75$ (route 31 today, AM). Capacity $= 60 \times 0.75 = 45$ pax/h. $o \geq 129.5 - 45 = 84.5$ and $o \geq 0$, so $o = 84.5$ pax/h. If instead $f = 3$, capacity is 180, $o \geq -50.5$ and $o \geq 0$, so $o = 0$.

10.3 Trick 3: "at least $f\tau$ buses" with an integer

Constraint C2 is $\tau \cdot f \leq x$ with x an integer. This automatically means $x \geq \lceil f\tau \rceil$ (Chapter 8). No rounding function is needed.

10.4 Trick 4: "close to today" as simple bounds

Rule C5a says the number of buses should not move too far from today's x_0: $|x - x_0| \leq \text{dev}$. The absolute value is not linear, but it is the same as two linear inequalities $x_0 - \text{dev} \leq x \leq x_0 + \text{dev}$. The project writes these directly as lower and upper bounds on x.

Worked example: practicality bounds

$\text{dev} = \max(1, \min(3, 0.3x_0))$ (decision D25).

1. $x_0 = 4$: $0.3 \cdot 4 = 1.2$; $\min(3, 1.2) = 1.2$; $\max(1, 1.2) = 1.2$. Bounds $[\lceil 2.8 \rceil, \lfloor 5.2 \rfloor] = [3, 5]$.
2. $x_0 = 20$: $0.3 \cdot 20 = 6$; $\min(3, 6) = 3$. Bounds $[17, 23]$.
3. $x_0 = 2$: 0.6; $\max(1, 0.6) = 1$. Bounds $[1, 3]$.

10.5 Trick 5: a soft rule as a penalty

Rule C5b (equity) says no headway should exceed a cap (60 min in the peaks, 90 at midday). Some published headways are 75–90 min, so a hard rule would make the published-bounds model infeasible (decision D26). Instead the *excess minutes* are added to the objective as a penalty:

$$\text{penalty} = \sum_{r,t,k} \max(0, h_k - \text{cap}_t)\, y_{rtk}.$$

For each menu option the excess $\max(0, h_k - \text{cap})$ is a fixed number, so this is linear.

Worked example: equity excess

Peak cap 60 min. Options 45, 60, 80 min have excess 0, 0, 20. Choosing 80 adds 20 minutes of penalty.

10.6 Trick 6: tie-breaks and locks

- **Tie-break.** When many schedules have the same overflow, Stage 3 adds tiny terms $10^{-6} Z_2 + 10^{-4} \sum x$ so the solver picks the cheapest of them (`TIEBREAK` in `src/stage3.py`).
- **Lock.** After optimising one goal, add a constraint "this goal may not get worse than the value just found" before optimising the next (Chapter 15). The lock is itself a linear constraint.

Common confusion

These tricks do not approximate. Trick 1 restricts the choices to the menu (a real modelling decision: timetables use round headways anyway). Tricks 2–6 are *exact* rewrites: the linear model gives exactly the same answer as the nonlinear rule would.

How this appears in our project

The project's corrections of the original synopsis (`docs/MODEL.md`, "Corrections"):

1. Waiting $\sum D \cdot 30/f$ was nonlinear; now it is linear in the headway switches (Trick 1).
2. Operating cost was $d \cdot x$ (counts buses, not kilometres); now it is vehicle-km $d_{\text{cycle}} \cdot f \cdot H$.
3. Overflow used capacity $C \cdot x$; the correct hourly capacity is $C \cdot f$ (Trick 2).
4. Fuzzy demand used the lower α-cut; now it uses the upper one (Chapter 13).

Check your understanding

1. $D = 50$, $C = 60$, $f = 1$. What is o?

   Answer: $50 - 60 = -10$, so $o = 0$.

2. Write $|x - 6| \le 2$ as two inequalities.

   Answer: $4 \le x \le 8$.

3. Why is $\max(0, h_k - \text{cap})\, y_{rtk}$ linear?

   Answer: Because $\max(0, h_k - \text{cap})$ is a number computed in advance; it only multiplies the variable y.

Part II

The bus world

Chapter 11

How a bus network runs

11.1 Routes, terminals and loops

Definition: route, terminal, loop route

A **route** is a fixed path a bus follows, with a number such as 31 or 25/25A. Its two ends are **terminals**. A **loop route** starts and ends at the same place.

The CTU timetable used here (as on 25.07.2026) has **68 routes**, **389 buses**, **1662 km** of published route length and **649 stops**. Eight routes are loops: 2A, 2C, 4A, 4C, 5A, 5C, 9A, 9C (decision D2). Route numbers are stored as text, never as numbers, because "25/25A" is not a number and "1" must not become "1.0" (`src/data.py`).

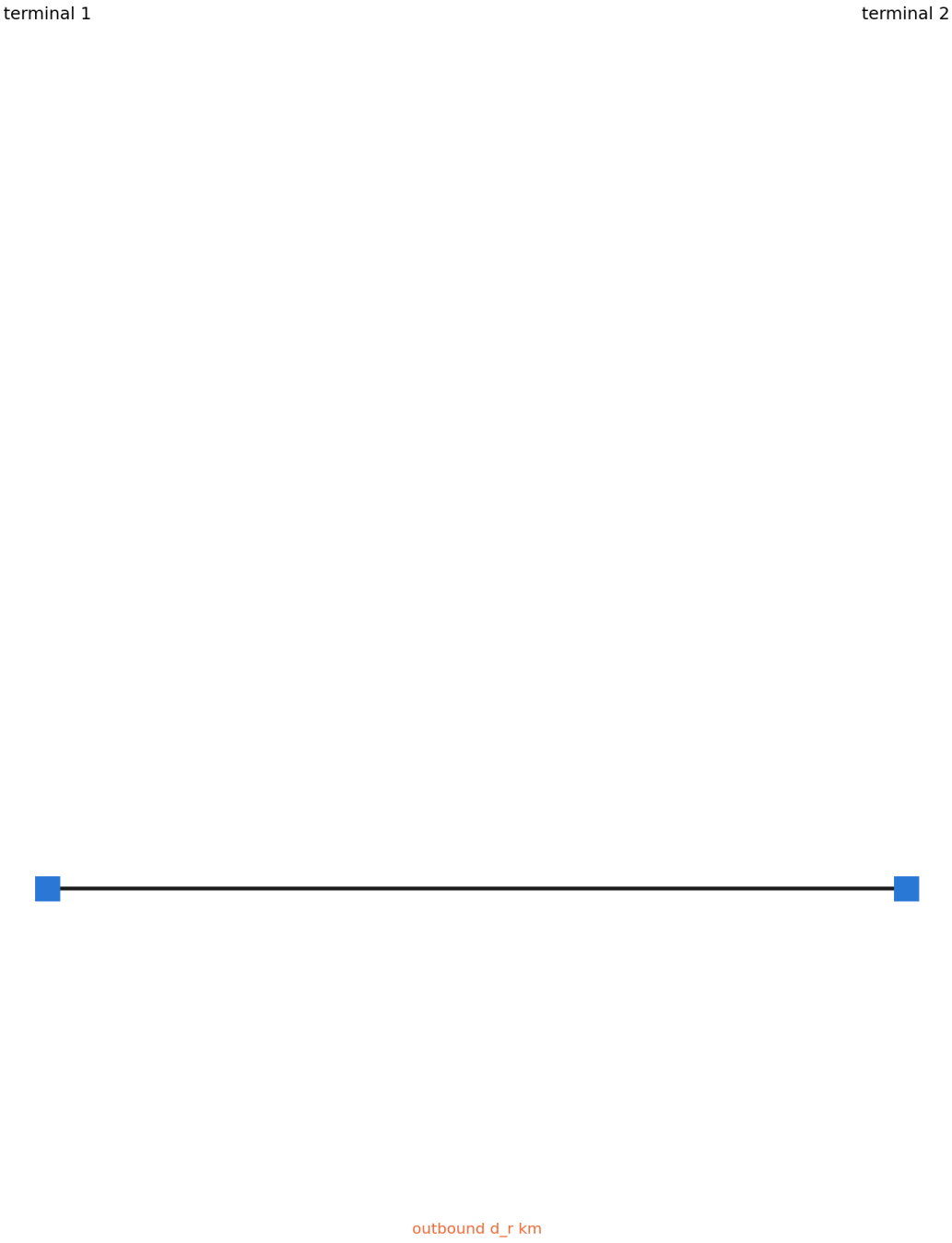
11.2 Cycle time

Definition: cycle time τ

The **cycle time** τ_r ("tau") is how long one bus takes to go out, come back, and be ready to leave again. It is the driving time plus a **layover** (a rest and recovery pause at the terminal):

$$\tau_r = \frac{L_r}{v} \times (1 + \ell),$$

with L_r the distance of one full cycle, v the average speed (including stops) and ℓ the layover as a fraction of driving time.


terminal 1
terminal 2
outbound d_r km

Definition: the three readings of route length: S1, S2, S3

The spreadsheet gives each route a "Route Length" d_r but does not say if it is one way or the whole trip. So the project tested three **scenarios** (`src/calibration.py`):

- **S1**: d_r is the full cycle, $L_r = d_r$.
- **S2**: d_r is one way for normal routes, so $L_r = 2d_r$; loops keep $L_r = d_r$.
- **S3**: S2, plus 10% spare buses kept at depots.

Worked example: cycle time of route 31

Route 31 (Omex Tower to Mansa Devi): $d_r = 37$ km, not a loop. Under S2 with $v = 22$ km/h and $\ell = 0.10$:

1. Full-cycle distance $L = 2 \times 37 = 74$ km.
2. Driving time $74/22 = 3.364$ h.
3. Add 10% layover: $\tau = 3.364 \times 1.10 = 3.70$ h.

11.3 Little's law: how many buses a frequency needs

Definition: buses needed

To run f departures per hour on a route whose cycle takes τ hours, you need

$$n = f \times \tau \text{ buses on the road.}$$

This is a form of **Little's law** (average number in a system = arrival rate × time spent in the system).

Intuition

A merry-go-round with seats coming past the gate every 10 minutes and a ride lasting 60 minutes needs $60/10 = 6$ seats in the ring. In bus terms: if a bus must leave every $1/f$ hours and each bus is away for τ hours, then $\tau/(1/f) = f\tau$ buses are out at any moment.

Worked example: route 31 needs 3 buses

Published headway 80 min, so $f = 60/80 = 0.75$ dep/h. Buses needed $= 0.75 \times 3.70 = 2.78$, so at least 3 whole buses. The timetable gives route 31 exactly 3 buses. The calibration is consistent.

11.4 Calibration: which reading of d_r is right?

Adding $f_{\text{mid}} \cdot \tau$ over all routes gives N_{need}, the buses the published timetable needs. Comparing with the 389 buses tests the assumptions (`outputs/calibration_report.md`, decision D42):

Reading (at 22 km/h)	layover	N_{need}	share of 389
S1 (full cycle)	0%	171.9	44%
S2 (one way)	0%	309.9	80%
S2 (one way)	10%	340.9	88%
S3 (S2 + 10% spares)	10%	375.0	96%

Under S1 more than half the fleet would sit idle and 57 of 68 routes would have more buses than even their highest frequency needs. That is implausible, so the project uses **S2, 22 km/h, 10% layover**.

11.5 Fleet, spares and the usable fleet

Definition: spare ratio and usable fleet

Some buses are always in maintenance or kept as reserves. With a spare ratio s, the buses that may be scheduled are

$$B_{\text{eff}} = \lfloor (1 - s)\, B \rfloor .$$

$\lfloor \cdot \rfloor$ means round down. The project uses $s = 0.08$: $B_{\text{eff}} = \lfloor 0.92 \times 389 \rfloor = \lfloor 357.9 \rfloor = 357$ (decision D22/D45).

Common confusion

Why not 10% spares? $\lfloor 0.9 \times 389 \rfloor = 350$, but the published *minimum* frequencies already need 352 buses in the AM peak. The model would be infeasible. The published timetable itself keeps 380–385 buses on the road, so the optimiser gets 28 *fewer* buses than today. Any improvement it finds is therefore "without extra vehicles".

11.6 Capacity, load factor and overflow

Definition: capacity, load factor, overflow

- A bus carries $C = 60$ passengers (seated plus standing; an assumption, decision D23).
- Hourly **capacity** of a route $= C \times f$ (passengers per hour past the busiest point).
- **Load factor** LF = demand/capacity. LF = 1 means exactly full; LF $>$ 1 means overcrowded; LF $<$ 0.3 is called under-used in the project.
- **Overflow** $= \max(0, \text{demand} - \text{capacity})$: people who do not fit.

Worked example: route 31, AM peak

Most likely demand $M = 110.2$ pax/h. Capacity $= 60 \times 0.75 = 45$ pax/h. LF $= 110.2/45 = 2.45$. This is the most overloaded route-period in the synthetic data (`outputs/demand_report.md`).

Common confusion

Capacity per hour is $C \times f$, *not* $C \times x$. Ten buses parked at a terminal carry nobody. What counts is how many buses *pass* per hour. This was one of the corrections made to the original

synopsis.

11.7 Vehicle-kilometres

Definition: vehicle-km

Vehicle-kilometres count the kilometres driven by all buses. One departure in each direction drives one full cycle d_{cycle}, so in H hours a route drives $d_{\text{cycle}} \times f \times H$ km.

Worked example: route 31, AM

$d_{\text{cycle}} = 74$ km, $f = 0.75$, $H = 3$ h: $74 \times 0.75 \times 3 = 166.5$ vehicle-km.

11.8 Time periods and partial service

The day is split into three periods (the spreadsheet's editable cells, decision D13): AM peak 07:00–10:00 ($H = 3$ h), Midday 10:00–16:00 (6 h), PM peak 16:00–19:00 (3 h). A route is **active** in a period if its service window overlaps it: 64 routes in AM, 65 at Midday, 62 in PM, so **191 active route-periods**. Some routes run only part of a period (route 11 runs 0.67 h of the PM peak), so the model counts only the overlap hours H_{rt} (decision D20).

How this appears in our project

- τ_r, d_{cycle}: `src/calibration.py`. Figure `calibration_N_need.png` (Chapter 19).
- C1 (fleet): $\sum_r x_{rt} \le 357$ in each period. C2 (Little's law): $\tau_r f_{rt} \le x_{rt}$.
- Overflow and capacity: constraint C3 and objective Z_3. Vehicle-km: objective Z_2.
- The published timetable breaks C1 (it uses up to 385 buses) and C2 on routes 40, 76 and 25/25A at 22 km/h (`outputs/stage3_report.md`).

Check your understanding

1. A loop route has $d_r = 20$ km. Its cycle time at 22 km/h with 10% layover?
   Answer: Loops keep $L = d_r$: $20/22 \times 1.1 = 1.0$ h.
2. That loop runs every 15 min. Buses needed?
   Answer: $f = 4$; $4 \times 1.0 = 4$ buses.
3. Demand 90 pax/h, headway 30 min, $C = 60$. Load factor and overflow?
   Answer: Capacity $= 60 \times 2 = 120$; LF $= 0.75$; overflow 0.
4. What is B_{eff} with $s = 0$?
   Answer: 389.

Chapter 12

Waiting time

12.1 Why the average wait is half the headway

Definition: random arrival

Passengers arrive **at random** if they do not time their arrival to the timetable: every moment between two buses is equally likely. This is realistic when buses are frequent (people just walk to the stop).

Worked example: the half-headway rule

Buses come every $h = 12$ min.

1. Someone arriving just after a bus waits almost 12 min.
2. Someone arriving just before the next bus waits almost 0 min.
3. Arrival times are spread evenly, so waits are spread evenly between 0 and 12.
4. The average of numbers spread evenly between 0 and 12 is the midpoint: $(0+12)/2 = 6$ min $= h/2$.



Figure 12.1: Teaching example with 14 random passengers. Top: regular buses, mean wait close to $h/2 = 6$. Bottom: the same six buses bunched in pairs, and the mean wait grows.

12.2 Demand-weighted waiting

Busy routes matter more: 300 people waiting 10 minutes is worse than 30 people waiting 10 minutes.

Definition: total waiting rate Z_1

$Z_1 = \sum_{r,t} M_{rt} \cdot \dfrac{h_{rt}}{2}$, in passenger-minutes per hour. M_{rt} is the most likely demand. Dividing Z_1 by total demand $\sum M_{rt}$ gives the average wait per passenger.

Worked example: weighted average by hand

Route A: 300 pax/h, headway 10 (wait 5). Route B: 100 pax/h, headway 30 (wait 15). $Z_1 = 300 \times 5 + 100 \times 15 = 3000$. Mean wait $= 3000/400 = 7.5$ min. The simple average of 5 and 15 is 10, but most people use route A, so the passenger average is 7.5.

12.3 Irregular buses wait longer: the $1 + \mathrm{CV}^2$ factor

Definition: coefficient of variation (CV)

The **CV** of the gaps between buses is their standard deviation divided by their mean. CV $= 0$ means perfectly regular; CV $= 0.3$ means gaps typically vary by about 30% of the mean.

Definition: expected wait with irregular gaps

For random arrivals, $E[W] = \dfrac{h}{2}(1 + \mathrm{CV}^2)$, where h is the mean gap. The extra part $\dfrac{h}{2}\mathrm{CV}^2$ is the **excess wait time** (EWT).

Worked example: why long gaps hurt

Buses alternate gaps of 2 min and 22 min (mean 12, like the bunched picture).

1. In each 24-minute block, a random person lands in the long gap with probability 22/24 and in the short gap with probability 2/24. Long gaps "catch" more people.
2. Inside a gap, the average wait is half the gap: 11 or 1 min.
3. Expected wait $= \frac{22}{24} \times 11 + \frac{2}{24} \times 1 = 10.08 + 0.08 = 10.17$ min.
4. Check with the formula: the gaps 2 and 22 have mean 12 and standard deviation 10, so CV $= 10/12 = 0.833$. $E[W] = 6 \times (1 + 0.694) = 10.17$ min. The same answer.

The same number of buses, but people wait 70% longer.

Common confusion

"If buses come every 12 minutes on average, I wait 6 minutes on average" is only true when they are regular. With bunching you are more likely to arrive during a long gap. This is called the *inspection paradox*.

How this appears in our project

- Z_1 in the model uses $h/2$ and the most likely demand M (`src/model.py`). The published baseline has $Z_1 = 184{,}777$ pax · min/h and a mean wait of 16.6 min.
- The simulation (Stage 6) uses gamma-distributed gaps with CV $= 0.3$, so waits are multiplied by $1 + 0.09 = 1.09$. Baseline expected wait in simulation: 17.94 min; P-BALANCED: 17.34 min (3.3% less).
- CV $= 0.3$ is an assumption (no GPS data exists, decision D27). The formula is checked against a direct simulation in `tests/test_simulate.py`.
- Limitation: bunching changes waiting in the simulation, but not capacity.

Check your understanding

1. Headway 20 min, regular. Average wait?

   Answer: 10 min.

2. Same headway, CV $= 0.3$. Expected wait and excess wait?

   Answer: $10 \times 1.09 = 10.9$ min; EWT 0.9 min.

3. Why does the model multiply each route's wait by its demand?

   Answer: So that the objective counts *passenger* waiting; a busy route's wait affects more people.

Chapter 13

Demand and its uncertainty

13.1 Why the demand is synthetic

Definition: synthetic data

Synthetic data are numbers generated by a stated formula instead of being measured. CTU has published its timetable but not how many people ride each route. So the project *invents* plausible demand with a transparent recipe, and labels every result "synthetic demand, illustrative" (decision D19).

Intuition

A flight simulator is not a real flight, but a pilot can still learn how the controls respond. The synthetic demand shows how the *method* responds; it does not say what will happen in Chandigarh.

13.2 The recipe

1. **Score each route**: longer routes, and routes through big stops, get more passengers.

   $$\text{score}_r = d_r \times (1 + 0.6\,\text{PGI} + 0.4\,\text{ISBT43} + 0.3\,\text{ISBT17} + 0.3\,\text{outskirts} + 0.3\,\text{Derabassi} + 0.05\,\text{hubs}) \times \text{noise}_r$$

   Each flag is 1 if the route serves that place and 0 otherwise. "hubs" counts stops served by at least 10 routes. noise_r is a random factor around 1 (LogNormal with $\sigma = 0.15$, seed 42).

2. **Scale to a believable city total**: multiply all scores by one number λ chosen so that the network has about 81,326 boardings per day. That target is 143,000 CTU-wide daily riders scaled by 389/684 buses (an **unverified** newspaper anchor). The scaled score is $D_{\text{ref},r}$, passengers per hour at the busiest section.

3. **Shape the day**: multiply by 1.4 in the AM peak, 0.8 at midday and 1.3 in the PM peak: $M_{rt} = m_t \times D_{\text{ref},r}$.

Worked example: route 31 from start to finish

Route 31: 37 km; serves PGI and ISBT-17; has an outskirts terminal; passes 5 hub stops; noise = 0.949.

1. Uplift $= 1 + 0.6 + 0.3 + 0.3 + 0.05 \times 5 = 2.45$.
2. Score $= 37 \times 2.45 \times 0.949 = 86.03$.

3. $\lambda = 0.9151$ (found by the code so the total equals 81,326 boardings). $D_{\text{ref}} = 0.9151 \times 86.03 = 78.72$ pax/h.
4. AM: $M = 1.4 \times 78.72 = 110.2$ pax/h. Midday: $0.8 \times 78.72 = 63.0$. PM: $1.3 \times 78.72 = 102.3$.

These match `data/processed/demand_rt.csv`.

13.3 Uncertainty as a triangle: fuzzy numbers

Definition: triangular fuzzy number (TFN)

A **triangular fuzzy number** (L, M, U) describes an uncertain quantity by three values: a lowest plausible L, a most plausible M and a highest plausible U. Its **membership** $\mu(D)$ says how plausible the value D is: 1 at M, falling in straight lines to 0 at L and at U.

In the project $L = 0.80\,M$ and $U = 1.35\,M$. The upper side is longer because surges (rain, events, exams) are bigger than dips (decision D17).

Worked example: route 31, AM

$M = 110.2$, $L = 0.8 \times 110.2 = 88.2$, $U = 1.35 \times 110.2 = 148.8$ pax/h. Membership of $D = 129.5$: on the right side, $\mu = (U - D)/(U - M) = (148.8 - 129.5)/38.6 = 0.5$.

13.4 The α-cut: turning a triangle into one planning number

Definition: α-cut

Pick a plausibility level α between 0 and 1. The **α-cut** is every demand with membership at least α. For a triangle it is the interval

$$[\,L + \alpha(M - L),\; U - \alpha(U - M)\,].$$

The project plans capacity for the **upper end**: $D_\alpha = U - \alpha(U - M)$.

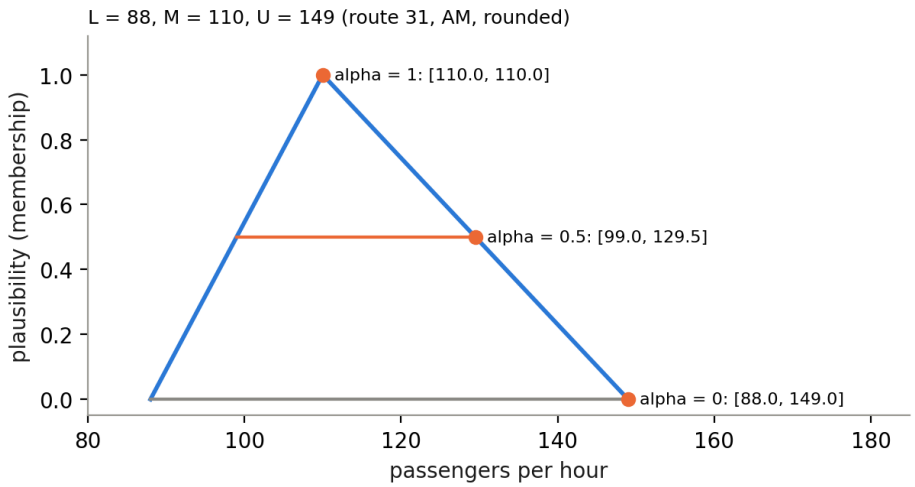


Figure 13.1: Horizontal slices of the triangle at three α levels. Orange dots are the planning demand D_α.

Worked example: three α values for route 31, AM

1. $\alpha = 1$: $D_1 = 148.8 - 1 \times 38.6 = 110.2 = M$ (plan for the most likely day).
2. $\alpha = 0.5$: $D_{0.5} = 148.8 - 19.3 = 129.5 = 1.175\,M$ (the project's default).
3. $\alpha = 0$: $D_0 = 148.8 = U$ (plan for the worst plausible day: most cautious).

Common confusion

Small α means cautious, not careless. $\alpha = 0$ uses the biggest demand. Also, the original synopsis used the *lower* end of the cut, which plans for *less* demand than likely and under-provides capacity; the project corrected it (decision D18).

Common confusion

Why M in waiting but D_α in overflow? Waiting Z_1 estimates the usual passenger experience, so it uses the most likely demand. Overflow is a safety requirement, so it plans for the high side (`docs/MODEL.md`).

13.5 How to read a heatmap

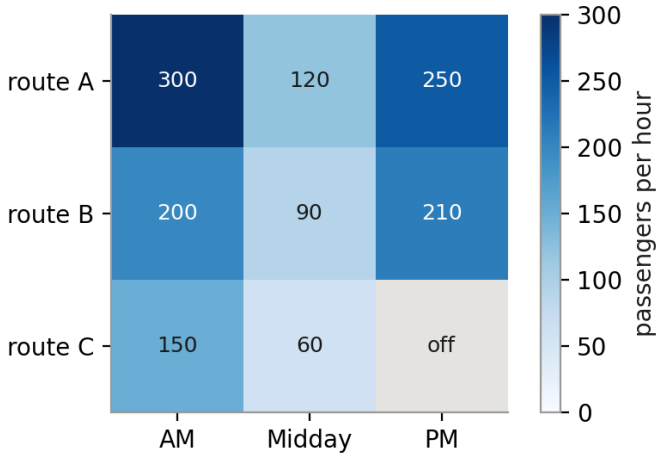


Figure 13.2: Teaching example: the three toy routes used in the notebook.

How to read: a heatmap

1. A heatmap is a table where each cell's *colour* shows its number. Rows and columns are labels (here routes and periods).
2. Find the colour bar. Here darker blue means more passengers per hour, white means close to 0.
3. Grey ("off") means no value: the route does not run then. It does *not* mean zero demand.
4. Read patterns, not single cells: here every route is darker in the peaks than at midday, because of the multipliers 1.4 / 0.8 / 1.3.

How this appears in our project

- Recipe: `src/demand.py` (`base_demand`, `modal_demand`, `add_tfn`, `alpha_cut_upper`). Settings: `src/params.py`.
- Results: total most-likely demand 4,480 pax/h (AM), 2,562 (Midday), 4,084 (PM). 24 route-periods are overloaded at today's capacity; 61 are under-used (`outputs/demand_report.md`).
- Robustness of the recipe: with 20 other random seeds the ranking of routes hardly changes (rank correlation 0.86–0.94).
- Figures: `demand_heatmap.png` and `tfn_alpha_cuts.png` (Chapter 19).

Check your understanding

1. $M = 40$. What are L and U?

   Answer: $L = 32$, $U = 54$.

2. With $M = 40$, what is $D_{0.5}$?

   Answer: $54 - 0.5 \times 14 = 47 = 1.175 \times 40$.

3. Which α is most cautious?

Answer: $\alpha = 0$.

4. Is the demand heatmap a measurement of CTU ridership?

   Answer: No; it shows the synthetic recipe's output.

Part III

Many goals at once

Chapter 14

Many objectives: domination and the Pareto front

14.1 Dominance

Definition: dominates

With objectives to minimise, solution A **dominates** solution B if A is *at least as good as* B on every objective *and strictly better* on at least one.

Worked example: who dominates whom

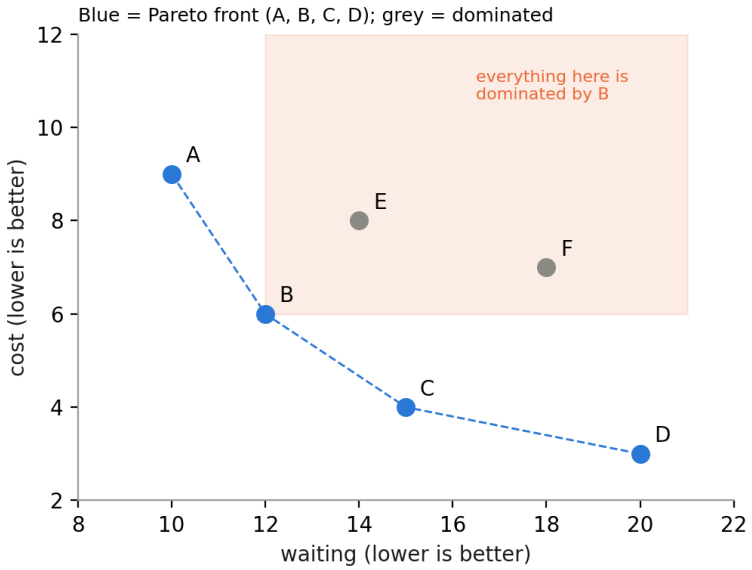
Teaching example, two objectives (waiting, cost), both lower-is-better: A $= (10, 9)$, B $= (12, 6)$, C $= (15, 4)$, D $= (20, 3)$, E $= (14, 8)$, F $= (18, 7)$.

1. B vs E: $12 \leq 14$ and $6 \leq 8$, strictly better on both. B dominates E.
2. B vs F: $12 \leq 18$, $6 \leq 7$. B dominates F.
3. A vs B: A waits less (10 vs 12) but costs more (9 vs 6). Neither dominates.
4. Is anything dominating A, B, C or D? Check each pair: no.

14.2 The Pareto front

Definition: Pareto-efficient, Pareto front

A solution is **Pareto-efficient** (non-dominated) if no feasible solution dominates it. The set of all of them, drawn in objective space, is the **Pareto front**. Named after Vilfredo Pareto, an Italian economist.



Intuition

Buying a phone where you care about price and battery life. A phone that is both dearer and has a worse battery than another is never worth buying: it is dominated. The front is the list of phones where, to gain battery, you must pay more. Choosing among them is a matter of taste, not of mathematics.

Common confusion

There is no single "optimal" point on a front. Every front point is optimal in the Pareto sense. Choosing one needs a preference: a target, a priority or a weight. That is what goal programming (Chapter 15) supplies.

14.3 Ideal and nadir

Definition: ideal and nadir

The **ideal** point takes the best value of each objective, found by optimising each one alone. It is usually not achievable: you cannot have the lowest waiting and the lowest cost at once. The **nadir** takes the *worst* value of each objective *among the Pareto-efficient solutions*. In practice it is estimated from the **payoff table**: optimise each objective alone, write down all objectives each time, and take the worst value in each column.

Worked example: payoff table of the project (Stage 5, relaxed $f_{\max}$, $\alpha = 0.5$)

minimised alone	Z_1 [pax · min/h]	Z_2 [veh-km/day]	Z_3 [pax/day]
Z_1	**175,794.9**	77,440.1	2,321.3
Z_2	193,752.9	**74,501.1**	3,286.4
Z_3	177,430.2	77,466.1	**2,173.1**

Ideal = the bold diagonal (175,794.9; 74,501.1; 2,173.1). Nadir = the worst of each column (193,752.9; 77,466.1; 3,286.4). Source: `outputs/pareto_report.md`.

14.4 Normalisation

Objectives have very different units and sizes ($Z_1 \approx 180{,}000$, $Z_3 \approx 2{,}000$). To compare them, rescale each to the range 0 (ideal) to 1 (nadir):

$$\tilde{Z} = \frac{Z - Z^{\text{ideal}}}{Z^{\text{nadir}} - Z^{\text{ideal}}}.$$

Worked example: normalising one value

$Z_3 = 2{,}729.8$: $\tilde{Z}_3 = (2729.8 - 2173.1)/(3286.4 - 2173.1) = 556.7/1113.3 = 0.50$, halfway between best and worst.

14.5 The knee point

Definition: knee

The **knee** is the front point closest (straight-line distance) to the ideal after normalisation. It is a common "balanced" pick. The project computes it in Stage 6 (`simulate.knee_point`).

14.6 Weighted sums and their weakness

A simple way to combine objectives is to minimise $w_1\tilde{Z}_1 + w_2\tilde{Z}_2 + w_3\tilde{Z}_3$. Each choice of weights picks one front point. Two weaknesses:

1. Without normalisation the biggest number wins: Z_1 would swamp Z_3.
2. For integer problems, some front points can never be found by any weights (they sit in "dents" of the front). The ε-constraint method of Chapter 16 fixes this.

How this appears in our project

- The project's three objectives: Z_1 waiting, Z_2 vehicle-km, Z_3 unserved.
- Stage 5 finds 46 exact front points (and 611 from NSGA-II); figures `pareto_2d.png` and `pareto_3d.png`.
- The published baseline is far from the front (normalised distance 1.093) and also breaks the fleet limit.

Check your understanding

1. Does $(5, 5)$ dominate $(5, 7)$?

   Answer: Yes: equal on the first, strictly better on the second.

2. Does $(4, 9)$ dominate $(5, 7)$?

   Answer: No: better on the first, worse on the second.

3. Why is the ideal point usually not achievable?

   Answer: Because the objectives conflict; the best waiting and the best kilometres come from different schedules.

4. Normalise $Z_2 = 76{,}000$.

   Answer: $(76000 - 74501.1)/(77466.1 - 74501.1) = 1498.9/2965 = 0.51$.

Chapter 15

Goal programming

15.1 Goals instead of "as good as possible"

Managers talk in targets: "cut waiting by 4%", "no route should wait more than an hour".

Definition: goal, target, deviation variables

A **goal** is a wish with a **target** value T. We add two new non-negative variables:

$$\text{achieved value} + n - p = T, \qquad n \geq 0,\ p \geq 0.$$

n ("negative" or under-achievement) is how far we land *below* the target; p (over-achievement) is how far we land *above* it.

Worked example: deviations by hand

Target waiting $T = 100$.

1. Achieved 90: $90 + n - p = 100$ gives $n = 10$, $p = 0$. We beat the target.
2. Achieved 115: $115 + n - p = 100$ gives $n = 0$, $p = 15$. We missed it by 15.

For a lower-is-better goal only p is unwanted. Goal programming minimises unwanted deviations, so a goal that is already met costs nothing.

Intuition

A student's goals: "at least 80% in maths" and "at most 2 hours of phone a day". Scoring 90% is not a problem; 3 hours of phone is 1 hour of unwanted deviation.

Common confusion

Goals are soft, constraints are hard. The fleet limit (357 buses) stays a hard constraint: it can never be broken. A goal such as "zero overflow" can be missed; the solver just makes the miss as small as it can.

15.2 The project's four goals

Targets are measured against the published timetable evaluated by the same model ("baseline"):

Goal	Meaning	Target	Unwanted
G1	waiting Z_1	0.96× baseline ($\kappa_1 = 0.04$)	p_1
G2	unserved Z_3	0	p_2
G3	vehicle-km Z_2	0.97× baseline ($\kappa_3 = 0.03$)	p_3
G4	equity: headway ≤ cap (60 / 90 / 60 min)	0 minutes above cap	$\sum p_4$

Worked example: G1 target in numbers ($\alpha = 0.5$)

Baseline $Z_1 = 184{,}776.6$. Target $T_1 = 0.96 \times 184{,}776.6 = 177{,}385.5$. P-BALANCED achieves 177,285.9: below the target, so $p_1 = 0$ and G1 is met (`outputs/gp_report.md`).

The targets were moved from 15% and 0% to 4% and 3% (decisions D48, D49) because the best possible waiting cut is only about 4.9%: a 15% target could never be reached, so it would act like plain "minimise".

15.3 Method 1: preemptive (priority) goal programming

Definition: preemptive / lexicographic GP

Rank the goals. Minimise the top goal's unwanted deviation. **Lock** it (add a constraint that it may not get worse). Then minimise the next goal, and so on. A lower goal can never improve by hurting a higher one.

Worked example: preemptive GP with three candidate schedules

Teaching example. Goal 1 (top): overflow. Goal 2: waiting. Candidates:

schedule	overflow miss	waiting miss
S	10	0
T	4	30
U	4	12

1. Level 1: the smallest overflow miss is 4 (T or U). Lock "overflow miss ≤ 4". S is now excluded, even though it is perfect on waiting.
2. Level 2: among T and U, the smallest waiting miss is 12. Choose U.

The project's three priority orders (`config/gp_scenarios.json`):

- **P-PASSENGER**: G2 > G1 > G4 > G3 (crowding, then waiting, equity, cost).
- **P-OPERATOR**: G3 > G2 > G1 > G4 (cost first).
- **P-EQUITY**: G4 > G2 > G1 > G3 (no long headways first).

15.4 Method 2: weighted goal programming

Definition: weighted GP

One solve: minimise $\sum_i w_i \, p_i / s_i$, where s_i is the baseline value of goal i. Dividing by s_i makes "1% worse" count the same for every goal.

Worked example: why divide by the baseline

Waiting misses by 1,000 pax · min/h on a baseline of 184,777: $1000/184777 = 0.0054$. Overflow misses by 100 pax/day on a baseline of 3,086: $100/3086 = 0.032$. Without dividing, waiting's 1,000 would look 10 times bigger; after dividing, overflow's miss counts about 6 times more, because it is the bigger *relative* miss.

The project's **P-BALANCED** uses equal weights $w_i = 1$. It is the headline policy (decision D51), because there is no evidence for ranking waiting above crowding or the reverse.

15.5 Method 3: Chebyshev (min-max) goal programming

Definition: Chebyshev GP

Minimise the *largest* normalised miss: add a variable λ with $w_i p_i / s_i \leq \lambda$ for every goal, and minimise λ. No goal is left far behind.

Worked example: min-max by hand

Two schedules with normalised misses (0.10, 0.00) and (0.04, 0.05). Weighted sums: 0.10 and 0.09. Largest miss: 0.10 and 0.05. Chebyshev picks the second (largest miss 0.05).

15.6 The final "tidy" solve

Buses do not appear in Z_1–Z_3, so many schedules with equal goals could use different numbers of buses. After every method the project locks all goal results and minimises the total number of buses (decision D31). The reported fleet is therefore the smallest that achieves those goals.

15.7 Results in the project ($\alpha = 0.5$, relaxed $f_{\max}$)

scenario	ΔZ_1	ΔZ_2	ΔZ_3	equity excess [min]	goals met
P-PASSENGER	−4.0%	−2.5%	−29.6%	100	0 of 4
P-OPERATOR	−3.8%	−3.0%	−29.6%	100	1 (G3)
P-EQUITY	−4.0%	−3.1%	−26.4%	0	3 (G1, G3, G4)
P-BALANCED	−4.1%	−3.1%	−26.4%	0	3 (G1, G3, G4)
CHEBYSHEV	−3.8%	−3.0%	−29.6%	100	0 of 4

All use 357 buses (today: 385). G2 (zero overflow) is never met, on purpose: zero overflow is impossible with the fleet available, so G2 acts as "as little overflow as possible". Source: `outputs/gp_report.md`, `outputs/report_facts.json`.

Common confusion

"P-PASSENGER meets 0 goals, so it is bad"? No. It misses G1 by only 44.7 pax · min/h, because it first squeezes overflow to its lowest possible value (2,173 pax/day). "Goals met" counts targets reached, not quality.

Common confusion

With the *published* frequency limits, all scenarios give nearly the same answer (waiting +0.8 to +1.0%, overflow only −0.6%). The overloaded routes are already at their highest allowed frequency, so there is nothing to trade. That is why the headline uses "relaxed $f_{\max}$": each route may go up to two menu levels shorter than its shortest published headway (decision D50). This would need CTU's approval.

How this appears in our project

Code: `src/goals.py` (goal model, the three methods, lock, tidy, payoff table) and `src/stage4.py` (runs 5 scenarios × 5 α values × 2 $f_{\max}$ modes = 50 runs; checks that every preemptive run respects its priority order). Figure: `gp_pct_change.png`.

Check your understanding

1. Target 50, achieved 47. What are n and p?

   Answer: $n = 3$, $p = 0$.

2. In preemptive GP, can a level-3 goal improve by making level 1 worse?

   Answer: No; level 1 is locked.

3. What does P-BALANCED minimise?

   Answer: The sum of the four unwanted deviations, each divided by its baseline value, with equal weights.

4. Why the tidy solve?

   Answer: To report the smallest fleet that achieves the goal results, since buses are not in the objectives.

Chapter 16

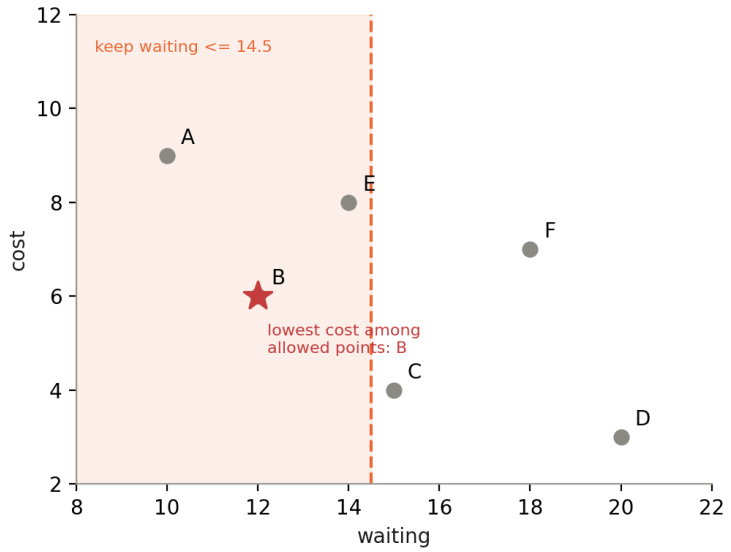
Computing a Pareto front: exact method and genetic algorithm

The project finds the front twice, with two completely different methods, and checks one against the other.

16.1 The ε-constraint method

Definition: ε-constraint

Keep one objective as the thing to minimise. Turn the others into constraints "at most ε". Solving for many different ε values traces out the front, one point per solve.



Worked example: one ε step by hand

Points from Chapter 14. Minimise cost subject to waiting ≤ 14.5.

1. Allowed (waiting ≤ 14.5): A $(10, 9)$, B $(12, 6)$, E $(14, 8)$.
2. Lowest cost among them: B with 6. So this solve returns B.
3. With waiting ≤ 16 we would also allow C $(15, 4)$, and C (cost 4) would be returned.

Moving ε step by step walks along the front: A, B, C, D.

16.2 AUGMECON: two refinements

The project uses **AUGMECON** (augmented ε-constraint, Mavrotas 2009):

1. **Augmentation.** Plain ε-constraint can return a "weakly" efficient point: one that ties on the main objective but is beaten on the constrained ones. AUGMECON adds a tiny reward for slack:

$$\min \ \frac{Z_3}{r_3} - 0.001\left(\frac{s_1}{r_1} + \frac{s_2}{r_2}\right) \quad \text{with} \quad Z_1 + s_1 = e_1,\ Z_2 + s_2 = e_2,\ s_1, s_2 \geq 0.$$

   s_1 and s_2 are how far below the limits we are; r_i are the ranges (nadir − ideal). Among equal-Z_3 schedules, the one with more slack (better Z_1, Z_2) wins, so every point is truly efficient.

2. **Grid with early exit.** e_1 and e_2 each take 20 values between nadir and ideal: a $20 \times 20 = 400$-cell grid. For a fixed e_1, e_2 goes from loose to tight; once a cell is infeasible, every tighter one is too, so they are skipped.

How this appears in our project

Stage 5 result (`outputs/pareto_report.md`): 298 cells optimal, 19 infeasible, 83 skipped. After removing duplicates and dominated points: **46 exact efficient points** in about 60 s. The infeasible corner is where both waiting and vehicle-km are pushed to their best at once, which cannot happen.

16.3 NSGA-II: a genetic algorithm

Definition: genetic algorithm

A **genetic algorithm** imitates evolution. It keeps a **population** of candidate solutions, mixes good ones (**crossover**), makes small random changes (**mutation**) and keeps the best (**selection**), for many **generations**. It does not prove optimality, but it can handle objectives no MILP can.

Definition: NSGA-II

The **Non-dominated Sorting Genetic Algorithm II** (Deb et al., 2002) is a genetic algorithm for several objectives. It ranks solutions in "fronts" (rank 1 = non-dominated, rank 2 = non-dominated once rank 1 is removed, and so on), and among equal ranks prefers solutions in less crowded parts of the front (**crowding distance**), so the population spreads along the whole front.

Worked example: the encoding, crossover and mutation

Each solution is a list of 191 numbers (**genes**), one per active route-period. A gene is the position of the chosen headway in that route's sorted menu (0 = shortest). Teaching example with 5 genes:

parent 1	2 0 3 1 4
parent 2	1 1 2 2 0
uniform crossover (coin flip per gene: swap at genes 2 and 4)	2 1 3 2 4
mutation (gene 5 reset to a random option)	2 1 3 2 1

Definition: repair

A random genome may need more than 357 buses. The project **repairs** it: while a period uses too many buses, lengthen the headway on the route-period where releasing a bus costs the least (smallest normalised loss in waiting, overflow and km per bus released). The repaired genome replaces the original ("Lamarckian" repair).

Intuition

Ordering food for a group within a budget. If the bill is too high, swap the dish whose downgrade the group will notice least, and repeat until the bill fits.

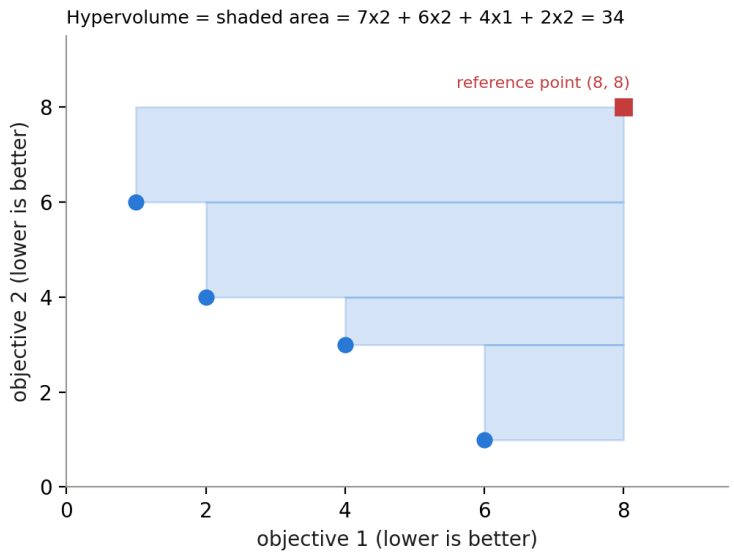
How this appears in our project

Settings (decision D35): population 200, 300 generations, crossover probability 0.9, mutation probability 1/191 per gene, 5 random seeds. Each seed evaluates $200 + 300 \times 200 = 60{,}200$ schedules in about 24 s. The five seeds together give 611 non-dominated points. Code: `src/pareto.py` using the DEAP library.

16.4 Measuring the quality of a front

Definition: hypervolume (HV)

In normalised space (0 = ideal, 1 = nadir), the **hypervolume** is the area (in 3-D, volume) dominated by the front and bounded by a reference point (the project uses 1.1 in every objective). Bigger is better.



Worked example: hypervolume by hand

Front $(1,6),(2,4),(4,3),(6,1)$, reference $(8,8)$. Cut the shaded area into strips (see figure): $7\times 2+6\times 2+4\times 1+2\times 2=14+12+4+4=34$.

Definition: GD, IGD, coverage, strict dominance

- **GD** (generational distance): average distance from each NSGA-II point to the nearest exact point. Small means NSGA-II stays close to the true front.
- **IGD** (inverted GD): average distance from each exact point to the nearest NSGA-II point. Small means NSGA-II covers the whole front.
- **Coverage** $C(A,B)$: the share of B's points that some point of A matches or beats.
- **Strict dominance count**: how many exact points NSGA-II *strictly* beats. It must be 0; otherwise the "exact" front would not be exact. The run stops with an error if it is not 0.

	points	hypervolume	GD	exact points beaten
AUGMECON (exact)	46	0.946	0	–
NSGA-II, per seed	187–194	0.965–0.969	0.048–0.050	0
NSGA-II, 5 seeds together	611	0.980	0.049	0

Common confusion

"NSGA-II has a higher hypervolume, so it is better" is wrong here. Its hypervolume is higher because it has many more points filling the gaps between the 46 grid points, not because its points are better. It beats none of the exact points, and stays close to them (GD ≈ 0.05). The two methods agree; the exact method also proves optimality.

16.5 Why use both methods?

- The exact method gives points with a proof of optimality and lists the infeasible regions.
- NSGA-II needs no linearisation, so it could use objectives a MILP cannot handle (for example waiting taken from a bunching simulation). Agreement between the two is evidence that both are implemented correctly.

Check your understanding

1. In the ε example, what does "waiting ≤ 11" return?

   Answer: Only A is allowed, so A.

2. Why does AUGMECON add $-0.001(s_1/r_1+s_2/r_2)$?

   Answer: To prefer, among ties on Z_3, the schedule with better Z_1 and Z_2, so returned points are strictly efficient.

3. What does a strict dominance count of 0 show?

   Answer: That the genetic algorithm found nothing better than any exact point, consistent with the exact points being truly efficient.

4. Compute the hypervolume of the single point $(2,2)$ with reference $(8,8)$.

Answer: $6 \times 6 = 36$.

Chapter 17

Simulation, statistics and sensitivity

17.1 Random variables and distributions

Definition: random variable, distribution

A **random variable** is a quantity whose value is uncertain until observed, such as tomorrow's demand on route 31. Its **distribution** says which values are likely.

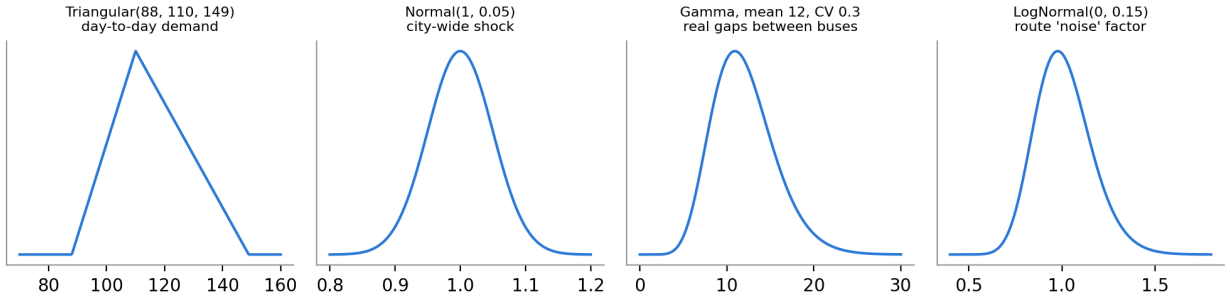


Figure 17.1: The four distributions the project uses. Higher curve = more likely value.

- **Triangular**(L, M, U): the TFN of Chapter 13 used as a probability shape. Each day's demand per route-period is drawn from it.
- **Normal**$(1, 0.05)$: a city-wide "shock" factor per day (rain, events), the same for all routes that day.
- **Gamma** with mean h and CV 0.3: the real gaps between buses.
- **LogNormal**$(0, 0.15)$: the fixed noise factor in the synthetic demand recipe.

17.2 Seeds: making randomness repeatable

Definition: random seed

Computers make "random" numbers with a formula started from a **seed**. The same seed always gives the same numbers. The project fixes its seeds (42 for demand noise, 2026 for the simulation, 1–5 for NSGA-II), so anyone who reruns it gets the same results.

17.3 Monte Carlo simulation

Definition: Monte Carlo

A **Monte Carlo simulation** draws many random scenarios, computes the result for each, and studies the spread of results. Named after the casino.

Worked example: one simulated day, one route

Route 31 AM under today's timetable (capacity 45 pax/h, $H = 3$ h):

1. Draw demand from Triangular(88.2, 110.2, 148.8): say 120.0 pax/h.
2. Draw the city shock from Normal(1, 0.05): say 1.03. Demand $= 120.0 \times 1.03 = 123.6$ pax/h.
3. Overflow $= \max(0, 123.6 - 45) \times 3 = 235.8$ passengers that day in that period.

The project does this for all 191 route-periods, for 2000 days.

Definition: common random numbers

Every timetable is tested on *the same* 2000 days. Then a difference between two timetables comes from the timetables, not from luck in the draws.

17.4 Summarising many results

Definition: mean, standard deviation, percentile

- **Mean**: the average.
- **Standard deviation (sd)**: the typical distance from the mean.
- **p-th percentile**: the value below which p% of results fall. The median is the 50th.

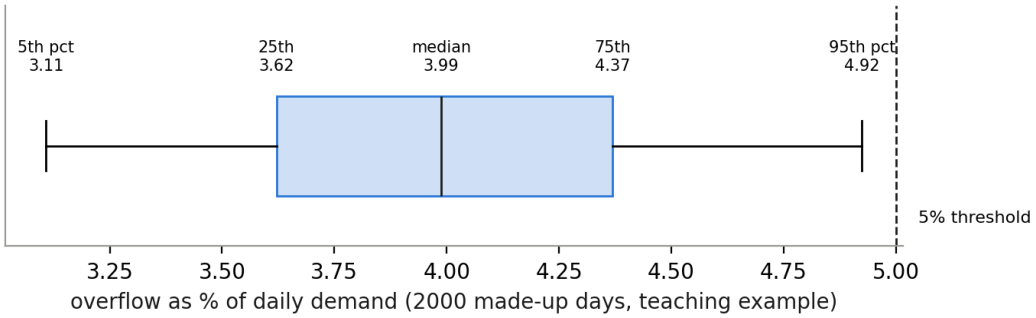
Worked example: percentiles of ten numbers

Sorted overflow shares (%): 3.1, 3.4, 3.6, 3.8, 3.9, 4.0, 4.2, 4.5, 4.9, 5.6. The median is between the 5th and 6th values: 3.95. One of ten values (10%) is above 5%: the estimated probability of "overflow above 5%" is 0.10.

Definition: exceedance probability

$P(\text{overflow} > 5\% \text{ of demand})$ is the share of simulated days on which more than 5% of passengers did not fit. It answers "how often is a day bad?".

17.5 How to read a box plot



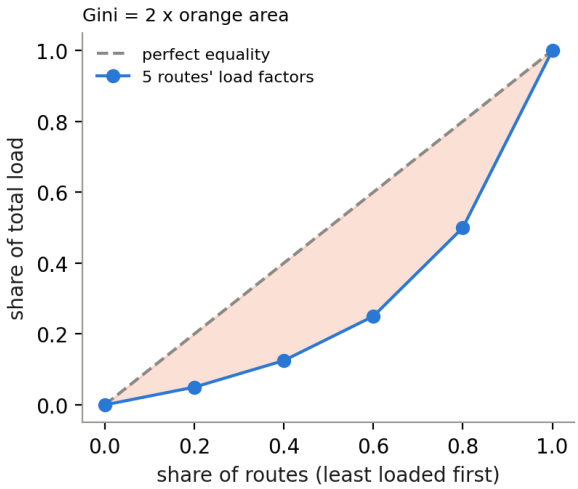
How to read: a box plot

- The box covers the middle half of the results (25th to 75th percentile). The line inside is the median.
- The whiskers (in this project) reach the 5th and 95th percentiles: 90% of days lie between them.
- A dashed threshold line lets you see at a glance how often results cross it.
- Compare boxes: a box further left (less overflow) and narrower (more predictable) is better.

17.6 The Gini coefficient

Definition: Gini

The **Gini coefficient** measures inequality: 0 means every route is equally loaded, values towards 1 mean load is concentrated on a few routes. It is twice the area between the diagonal and the *Lorenz curve*.



Worked example: Gini of five load factors

Load factors 0.2, 0.3, 0.5, 1.0, 2.0 (total 4.0). The project's formula (`src/metrics.py`, `gini`) is $G = \sum_i (2i - n - 1)v_i / (n \sum v)$ with the values sorted, $i = 1..n$: $[(-4)(0.2) + (-2)(0.3) + 0(0.5) + 2(1.0) + 4(2.0)]/(5 \times 4) = (-0.8 - 0.6 + 0 + 2 + 8)/20 = 8.6/20 = 0.43$.

17.7 Sensitivity analysis

Definition: one-at-a-time sensitivity, tornado chart

Change **one** assumption at a time (for example speed 18, 22, 26 km/h), re-optimise, and record how the result changes. A **tornado chart** draws one horizontal bar per assumption; the longest bars (at the top) are the assumptions the result depends on most.

Definition: out-of-sample test

Plan with one set of assumptions, then test on *different* data the plan did not see (here, demand 1.5 times more spread out and a stronger shock). It checks whether the plan only works on its own assumptions.

How this appears in our project

- Code: `src/simulate.py` and `src/stage6.py`. 2000 days, seed 2026, CV 0.3, shock sd 0.05.
- Headline (P-BALANCED, $\alpha = 0.5$): expected wait 17.34 vs 17.94 min (-3.3%); overflow 1,586 vs 2,249 pax/day (-29.5%); vehicle-km -3.1%; $P(\text{overflow} > 5\%) = 2.9\%$ vs 65.6%; 357 vs 385 buses.
- Sensitivity: speed matters most (26 km/h raises the waiting gain to 7.6%; 18 km/h is infeasible); layover 5% gives 5.9%, 15% is infeasible; no spares ($s = 0$) gives 3.7%; $s \geq 0.10$ is infeasible; demand level, bus capacity and α barely change the waiting gain.
- Out of sample: P-BALANCED's bad-day probability rises from 2.9% to 28.8%; the baseline's from 65.6% to 70.3%.

Check your understanding

1. Why fix random seeds?

   Answer: So that every rerun gives identical numbers (reproducibility).

2. Why test every timetable on the same 2000 days?

   Answer: So differences come from the timetables, not from different random draws.

3. In a tornado chart, what does the top bar tell you?

   Answer: Which assumption changes the result the most.

4. 130 of 2000 days have overflow above 5%. Probability?

   Answer: $130/2000 = 6.5\%$.

Part IV

Our project, assembled

Chapter 18

The complete model

This chapter puts every piece together. It is the mathematical heart of `src/model.py` and `docs/MODEL.md`.

18.1 Sets (what we loop over)

- Routes r: 68 routes.
- Periods $t \in \{\text{AM}, \text{Midday}, \text{PM}\}$.
- Active pairs (r, t): the route runs in that period. There are 191 (64 + 65 + 62).
- Menu K_{rt}: the headways route r may use in period t, taken from $\{10, 12, 15, 20, 25, 30, 40, 45, 60, 90\}$ plus the route's own published headways, and limited to the allowed range (C4).

18.2 Parameters (fixed numbers)

Symbol	Meaning and value	Unit
B_{eff}	usable fleet $\lfloor 0.92 \times 389 \rfloor = 357$	buses
C	bus capacity, 60	pax/bus
τ_r	cycle time $= L_r/22 \times 1.10$ (route 31: 3.70)	h
$d_{\text{cycle},r}$	km per cycle, $2d_r$ (non-loop) or d_r (loop)	km
H_{rt}	service hours of r inside period t (overlap)	h
M_{rt}	most likely demand (synthetic)	pax/h
$D_{\alpha,rt}$	planning demand $U - \alpha(U - M)$, $\alpha = 0.5$	pax/h
$x_{0,r}$	today's buses on route r	buses
$[\text{lb}_r, \text{ub}_r]$	"close to today" bounds, dev $=$ $\max(1, \min(3, 0.3x_0))$	buses
cap_t	equity headway cap: 60 (AM, PM), 90 (Midday)	min

18.3 Decision variables

For every active pair (r, t):

- $y_{rtk} \in \{0, 1\}$ for each headway k in the menu: 1 if that headway is chosen.

- $x_{rt} \in \{\text{lb}_r, \ldots, \text{ub}_r\}$, integer: buses.
- $o_{rt} \geq 0$: overflow passengers per hour.

Two helper expressions (not new variables): frequency $f_{rt} = \sum_k (60/h_k) y_{rtk}$ and headway $h_{rt} = \sum_k h_k y_{rtk}$.

18.4 Constraints

Name	Formula	In words, with a number
one headway	$\sum_k y_{rtk} = 1$	each route-period picks exactly one headway
C1 fleet	$\sum_r x_{rt} \leq 357$ for each t	at most 357 buses on the road in any period
C2 buses	$\tau_r f_{rt} \leq x_{rt}$	route 31 at 60 min: $3.70 \times 1 = 3.7$, so $x \geq 4$
C3 overflow	$o_{rt} \geq D_{\alpha,rt} - C f_{rt}$	route 31 AM at 60 min: $o \geq 129.5 - 60 = 69.5$
C4 range	built into the menu K_{rt}	route 31 (published 80 min) may use 45, 60 or 80 in "relaxed" mode
C5a near today	$\text{lb}_r \leq x_{rt} \leq \text{ub}_r$	route 31: $x_0 = 3$, bounds $[2, 4]$
C5b equity	soft penalty or goal G4	minutes above 60 (peak) or 90 (midday)
C6 inactive	no variables	a route not running gets nothing

18.5 Objectives

$$Z_1 = \sum_{r,t} M_{rt} \sum_k \frac{h_k}{2} y_{rtk} \qquad \text{waiting [pax} \cdot \text{min/h]}$$

$$Z_2 = \sum_{r,t} d_{\text{cycle},r}\, H_{rt}\, f_{rt} \qquad \text{vehicle-km [veh-km/day]}$$

$$Z_3 = \sum_{r,t} o_{rt}\, H_{rt} \qquad \text{unserved passengers [pax/day]}$$

Worked example: one route-period, every number

Route 31, AM peak, relaxed mode. Menu $\{45, 60, 80\}$. $\tau = 3.70$ h, $d_{\text{cycle}} = 74$ km, $H = 3$ h, $M = 110.2$, $D_{0.5} = 129.5$ pax/h, bounds $x \in [2, 4]$.

h	f	buses $\lceil f\tau \rceil$	allowed?	Z_1 part $Mh/2$	Z_2 part $74f \cdot 3$	Z_3 part $\max(0, 129.5 - 60f) \cdot 3$
80	0.75	3	yes	4,408	166.5	253.5
60	1.00	4	yes	3,306	222.0	208.5
45	1.33	5	**no** (ub = 4)	2,480	296.0	148.5

Going from 80 to 60 min costs one more bus and 55.5 km, and saves 1,102 pax · min/h of waiting and 45 overflow passengers. The 45-minute option is in the menu but needs 5 buses, more than C5a allows, so the solver can never pick it. The balanced plan picks 60 min for this route (the video's scene 5 shows the same numbers).

18.6 The different "runs"

Stage	What is minimised
Stage 3	Z_3 + soft equity penalty + tiny tie-breaks, with published $f_{\max}$ and then relaxed
Stage 4	goal deviations (preemptive, weighted, Chebyshev), then the tidy solve on $\sum x$
Stage 5	AUGMECON grid (minimise Z_3 with Z_1, Z_2 limits) and NSGA-II on (Z_1, Z_2, Z_3)
Stage 6	nothing new: the schedules are simulated

Model size: 191 x, 191 o, and 269 y switches with published limits (651 variables, 576 constraints); 633 switches with relaxed limits (1,015 variables, 576 constraints). Constraints: $191 \times 3 + 3 = 576$.

Common confusion

Why is the published timetable "infeasible" for our model? It keeps up to 385 buses on the road, more than 357, and three routes (40, 76, 25/25A) do not have enough buses for their own mid frequency at 22 km/h. It is still *evaluated* with the same formulas, as a baseline for comparison. Stage 3 lists its violations in `outputs/tables/baseline_violations.csv`.

How this appears in our project

- Build: `build_instance` (parameters), `build_core` (variables, C1–C6, Z_1–Z_3), `build_model` (Stage 3 objective). Solve: `solve`. Evaluate any schedule: `src/metrics.py`, `evaluate` and `check_schedule`.
- Stage 3 result with published limits: minimising overflow only lowers Z_3 from 3,086 to 3,067 pax/day, because the crowded routes are already at their highest published frequency. With relaxed limits it reaches 2,210 pax/day (`outputs/stage3_report.md`).

Check your understanding

1. How many constraints come from C1?

   Answer: 3, one per period.

2. Route 31, AM, headway 60: verify the Z_3 part.

   Answer: $129.5 - 60 = 69.5$ pax/h; times 3 h = 208.5.

3. Why can't route 31 use 45 min?

Answer: It needs $\lceil 1.33 \times 3.70 \rceil = 5$ buses, but C5a allows at most 4.

4. Which variables are binary, integer, continuous?

   Answer: y binary, x integer, o continuous.

Chapter 19

How to read every figure in the project

All 15 figures are in `outputs/figures/`. Every one that shows demand-based results says "(synthetic demand)" in its title. For each figure: what it shows, how to read it, what good and bad look like, and the conclusion to draw.

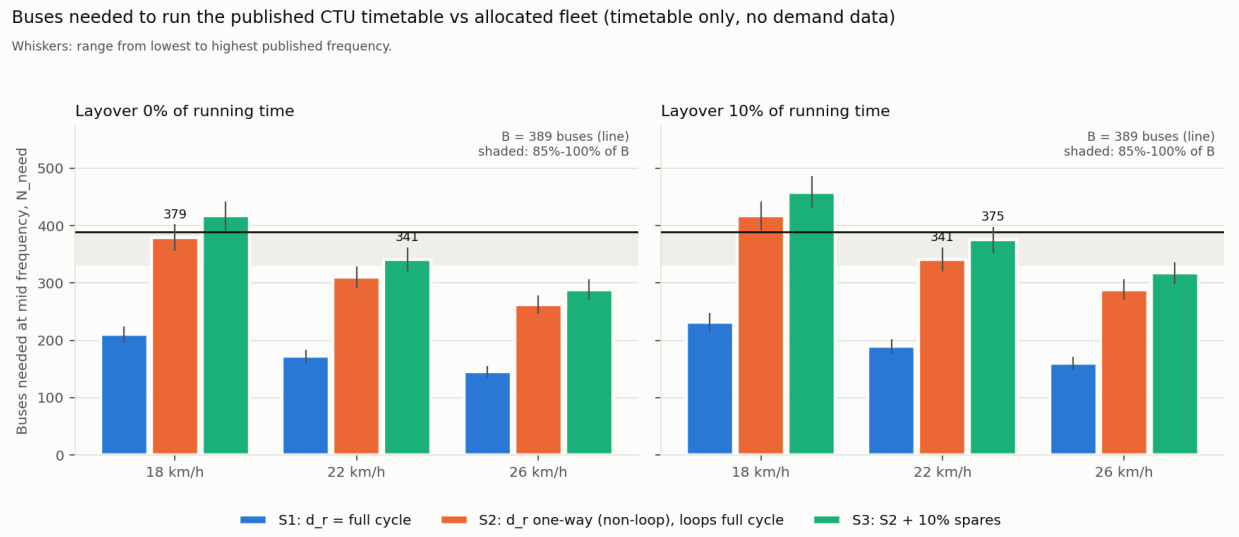
19.1 Block diagram of the method



How to read: methodology_block_diagram.png

Seven boxes, top to bottom, in the order the code runs: data audit, calibration, synthetic demand, MILP, goal programming, Pareto cross-check, simulation. Arrows mean "the output of this step feeds the next". Use it as your first slide: it is the map of the whole talk.

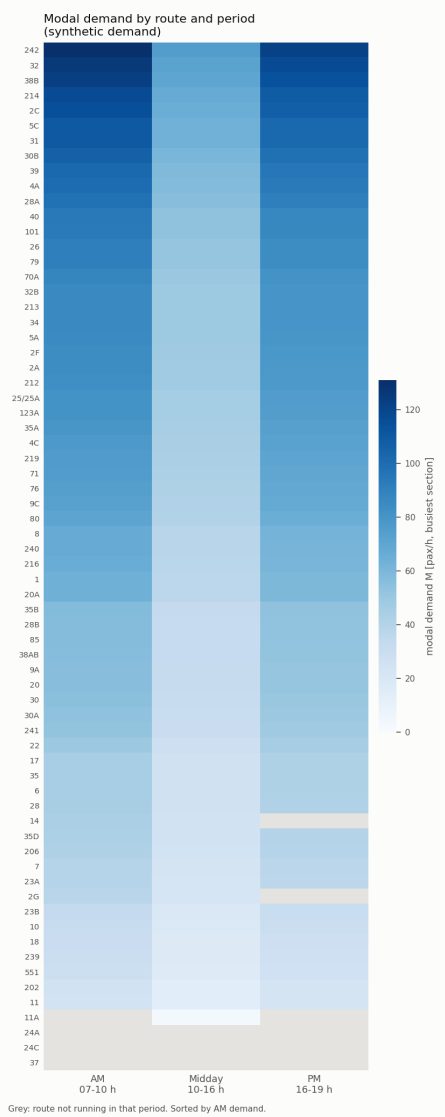
19.2 Calibration: buses needed vs fleet



How to read: calibration_N_need.png

- **Panels**: left with no layover, right with 10% layover. **Groups**: speeds 18, 22, 26 km/h. **Colours**: blue S1 (route length = full cycle), orange S2 (one way), green S3 (S2 + 10% spares).
- **Bar height**: buses needed to run the published mid frequency. **Whiskers**: from the lowest to the highest published frequency. **Black line**: the 389 buses CTU has. **Shaded band**: 85–100% of 389, a plausible level of use.
- **Good**: a bar inside the shaded band (the assumption explains how CTU uses its fleet). **Bad**: far below (half the fleet idle) or above the line (more buses than exist).
- **Conclusion**: blue bars are far too low, so S1 is wrong. The labelled bars (341 and 375 at 22 km/h, 10% layover) sit in the band. The project uses S2 at 22 km/h with 10% layover.

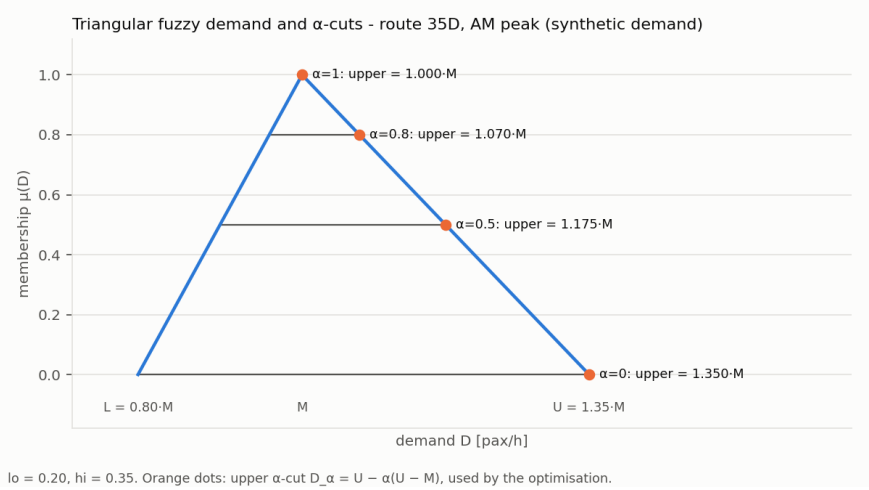
19.3 Demand heatmap



How to read: demand_heatmap.png

- **Rows**: the 68 routes, sorted by AM demand (busiest at the top). **Columns**: AM, Midday, PM.
- **Colour**: most likely demand M in passengers per hour at the busiest section; dark blue ≈ 130, white ≈ 0 (colour bar on the right). **Grey**: the route does not run in that period.
- **What to see**: every row is lighter at midday (multiplier 0.8 vs 1.4 and 1.3), and the long routes through PGI and the ISBTs (242, 32, 38B, 214) are darkest.
- **Conclusion**: this shows the synthetic recipe, not measured CTU ridership. It tells the audience where the model "thinks" demand is.

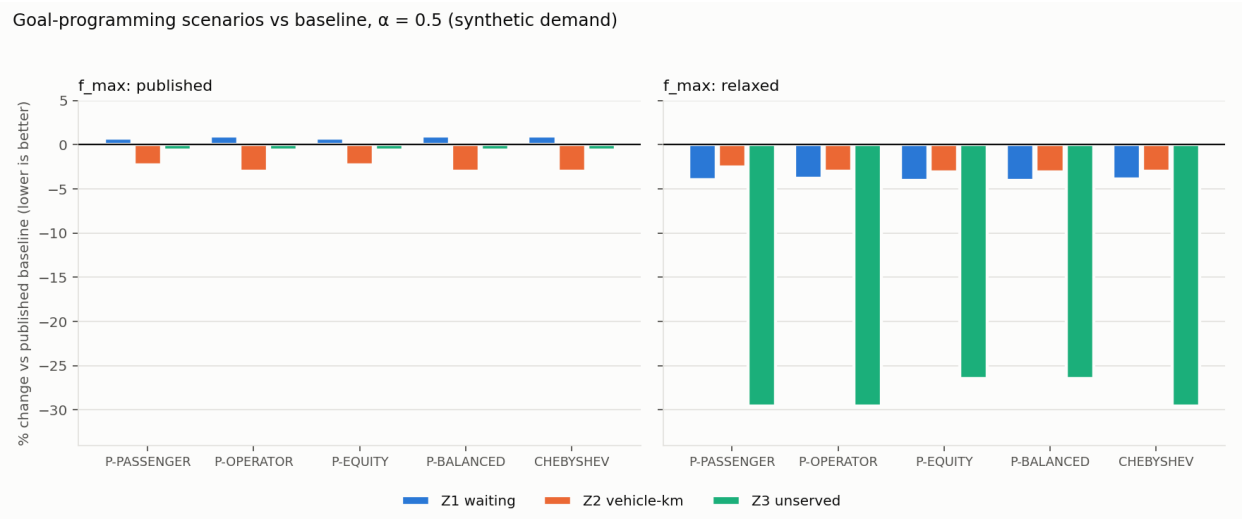
19.4 Fuzzy demand and α-cuts



How to read: tfn_alpha_cuts.png

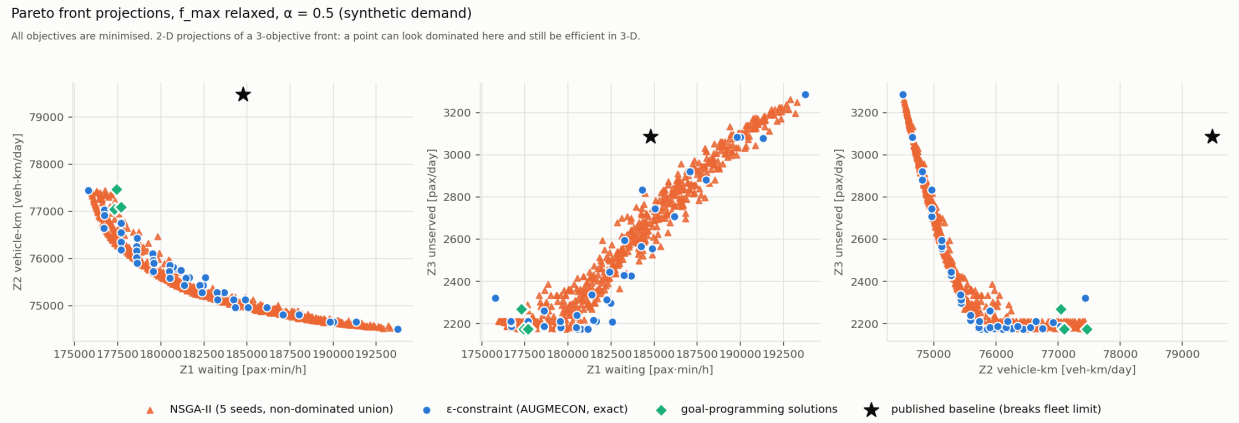
- **Across**: demand, relative to the most likely value M (route 35D, AM). **Up**: plausibility (0 to 1).
- The blue triangle runs from $L = 0.80M$ through M to $U = 1.35M$. Grey horizontal lines are α-cuts; orange dots are their upper ends, the demand used for capacity planning: $1.000M$, $1.070M$, $1.175M$, $1.350M$ for $\alpha = 1, 0.8, 0.5, 0$.
- **Conclusion**: lower α = plan for more passengers = more cautious. The default $\alpha = 0.5$ plans for 17.5% above the most likely demand.

19.5 Goal-programming scenarios



- **Panels**: left keeps today's frequency limits ("published"), right allows up to two menu levels shorter headways ("relaxed"). **Groups**: the five scenarios. **Colours**: blue waiting Z_1, orange vehicle-km Z_2, green unserved Z_3.
- **Bar height**: % change against today's timetable. Below zero is better (all three are "less is better").
- **Conclusion**: with published limits almost nothing changes (waiting even rises 0.8–1%). With relaxed limits every scenario cuts unserved passengers by 26–30% and waiting by about 4%, and vehicle-km by 2.5–3%. The priority order only shifts the result a little.

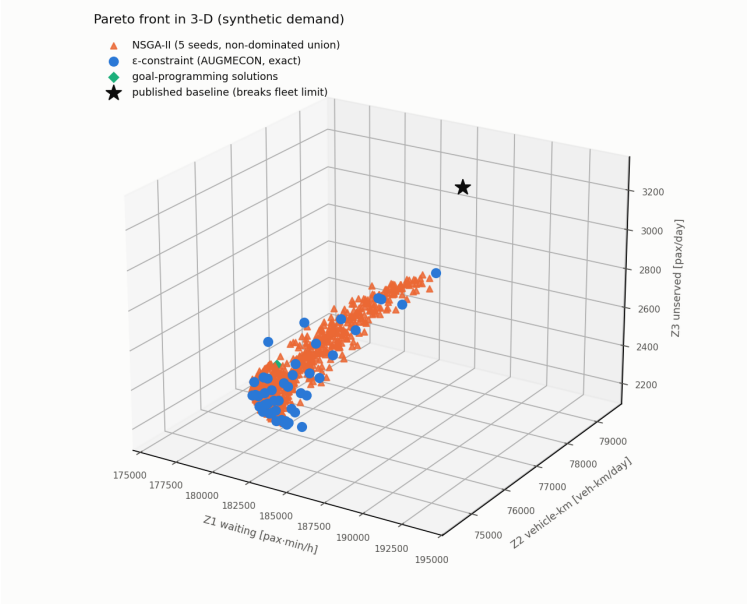
19.6 Pareto front, 2-D projections



How to read: pareto_2d.png

- Three scatter plots, each showing two of the three objectives: waiting–km, waiting–unserved, km–unserved. All axes are "lower is better".
- **Markers**: orange triangles NSGA-II (611 points), blue dots exact AUGMECON (46 points), green diamonds goal-programming solutions, black star today's timetable.
- **Shape**: the left panel's downward curve is the waiting–km trade-off: less waiting costs more km.
- **Good**: points towards the lower-left corner. **Bad**: the black star, far up and to the right (and it uses more buses than allowed).
- **Warning written on the figure**: these are 2-D shadows of a 3-D front, so a point may *look* dominated here while being efficient in 3-D.
- **Conclusion**: the two methods trace the same surface (they cross-check), the GP plans sit on or next to it, and today's timetable is far from it.

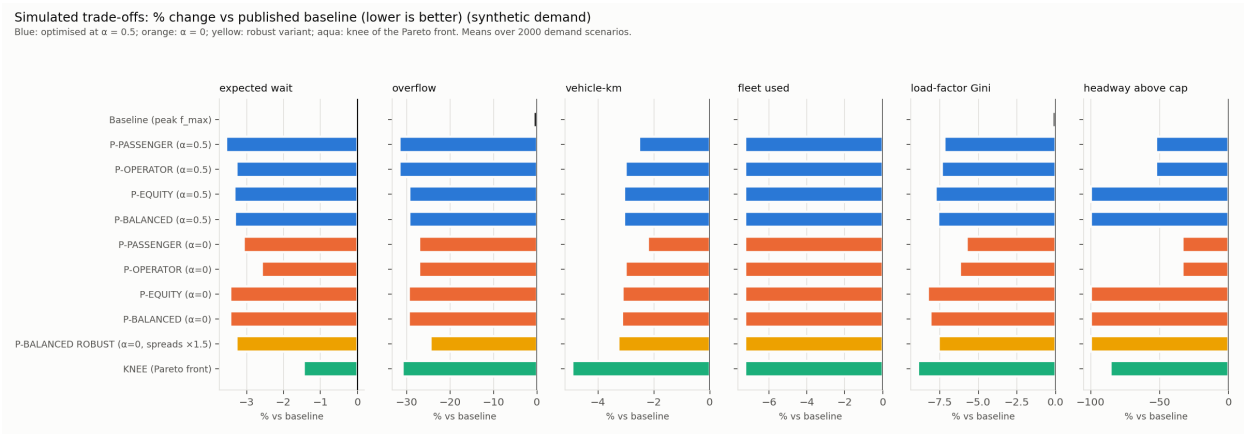
19.7 Pareto front in 3-D



How to read: pareto_3d.png

The same points in one 3-D picture: waiting (front axis), vehicle-km (right axis), unserved (up). The cloud is a thin curved sheet: that sheet is the front. The black star floats above and behind it. Use it to show the audience that "front" means a surface, then go back to the 2-D figure for details.

19.8 Simulated trade-offs

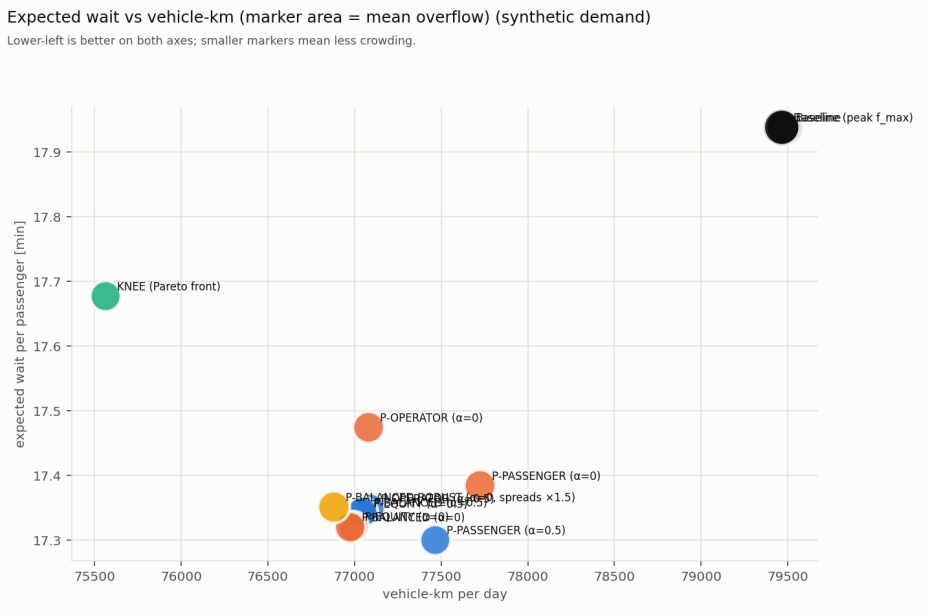


How to read: sim_tradeoff_bars.png

- Six small panels: expected wait, overflow, vehicle-km, fleet used, load-factor Gini, minutes of headway above the cap. One row per timetable. Bars show % change vs today; **left of zero is better**.
- **Colours**: blue = optimised at $\alpha = 0.5$, orange = at $\alpha = 0$, yellow = "robust" variant, aqua = knee of the Pareto front. Values are means over 2000 simulated days.

- **Conclusion**: every optimised timetable waits 1.5–3.6% less, overflows 24–32% less, drives fewer km, and uses 7.3% fewer buses. The knee drives the fewest km but waits the most of the optimised plans; P-EQUITY and P-BALANCED remove all minutes above the cap (−100%).

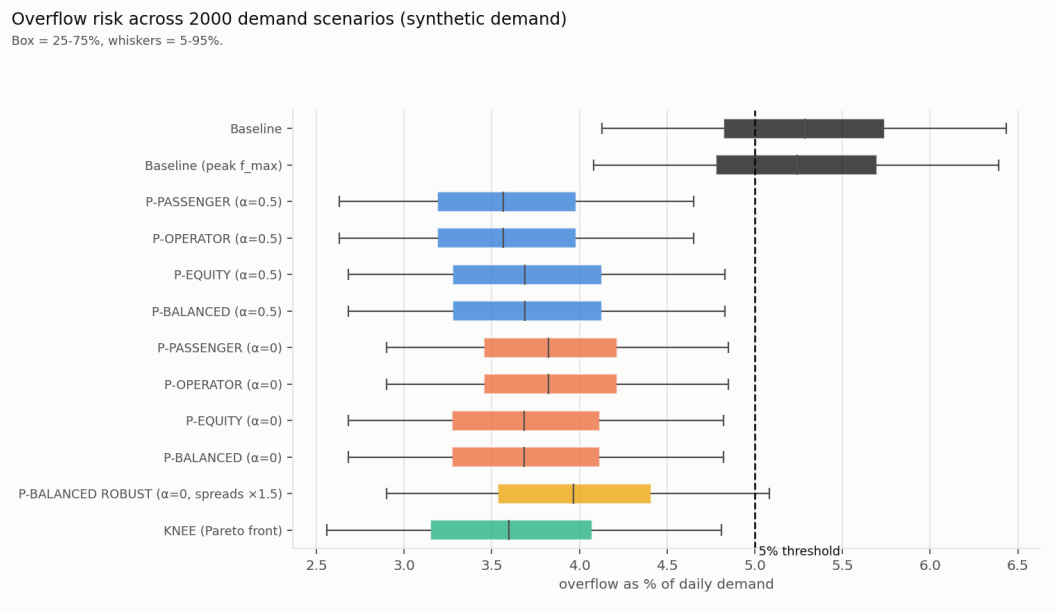
19.9 Wait vs vehicle-km



How to read: sim_wait_vs_vkm.png

- **Across**: vehicle-km per day. **Up**: expected wait per passenger. **Bubble size**: mean overflow. **Lower-left and small is better**.
- The black bubble (today) is top-right and biggest. The optimised plans cluster at about 17.3–17.5 min and 77,000 km. The aqua knee is far left (fewest km) but higher (longer wait).
- **Conclusion**: the optimised plans beat today on both axes at once; among themselves they trade a little waiting for a few kilometres. Note the vertical axis spans only 17.3–17.9 min: differences are small in minutes.

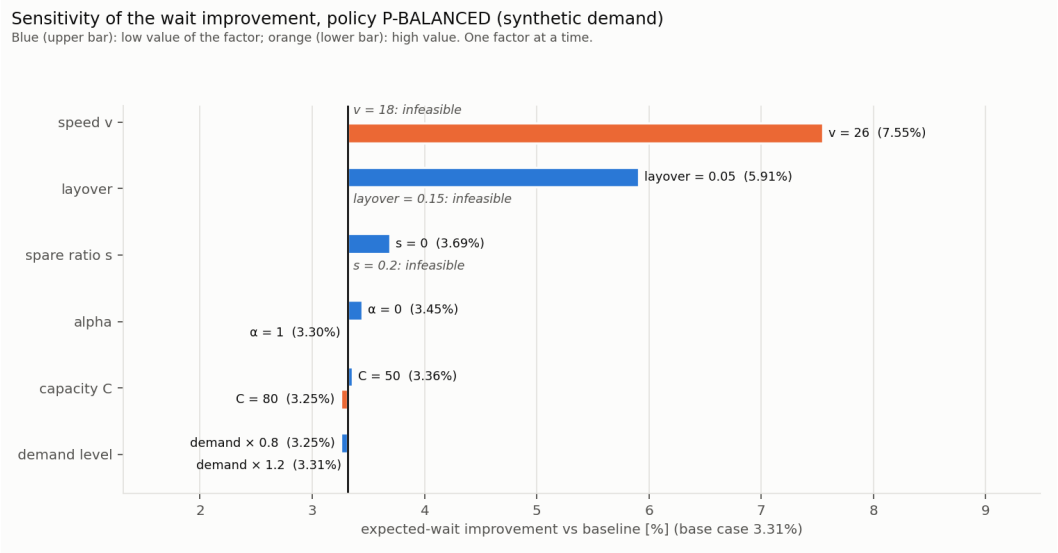
19.10 Overflow risk box plots



How to read: sim_overflow_distribution.png

- One box per timetable. **Across**: overflow as % of daily demand. Box = middle 50% of the 2000 days, line = median, whiskers = 5th to 95th percentile. Dashed line = the 5% threshold.
- **Good**: box far left of the dashed line. **Bad**: box across or right of it.
- **Conclusion**: today's timetable has its median above 5%: on most days (65.6%) more than 5% of passengers do not fit. All optimised plans sit around 3.5–4%, crossing 5% on only 1.5–6.6% of days.

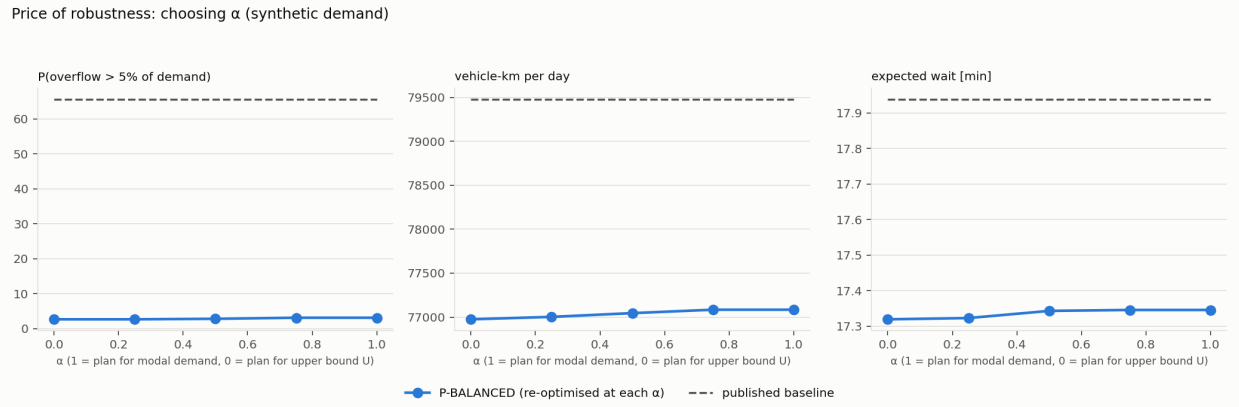
19.11 Tornado: sensitivity of the waiting gain



How to read: sensitivity_tornado.png

- **Vertical line**: the base result, 3.31% less waiting for P-BALANCED. Each row changes one assumption. Blue (upper bar) = low value, orange (lower bar) = high value. Text "infeasible" = no timetable exists.
- **Longest bars at the top** = assumptions that matter most.
- **Conclusion**: speed is decisive (26 km/h gives 7.55%; 18 km/h is infeasible), then layover (5% gives 5.91%; 15% infeasible), then spares ($s = 0$ gives 3.69%). Demand level, bus capacity and α hardly matter for the waiting gain. The 22 km/h and 10% layover assumptions are the ones to confirm with CTU.

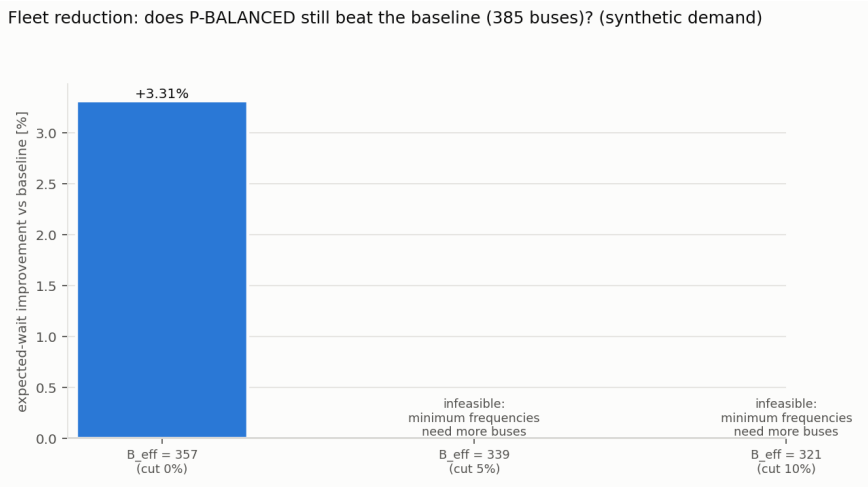
19.12 Price of robustness (α)



How to read: sensitivity_alpha.png

- Three panels, each with α across (0 = plan for the high demand U, 1 = plan for the most likely M). Blue line: P-BALANCED re-optimised at each α. Dashed: today.
- **Conclusion**: the blue lines are almost flat: bad-day probability 2.7–3.1%, vehicle-km 76,975–77,084, wait 17.32–17.35 min. With the fleet fixed, being more cautious changes little; all of them are far better than today (dashed line at 65.6%).
- Careful: each panel has its own vertical scale; the km and wait panels are zoomed in.

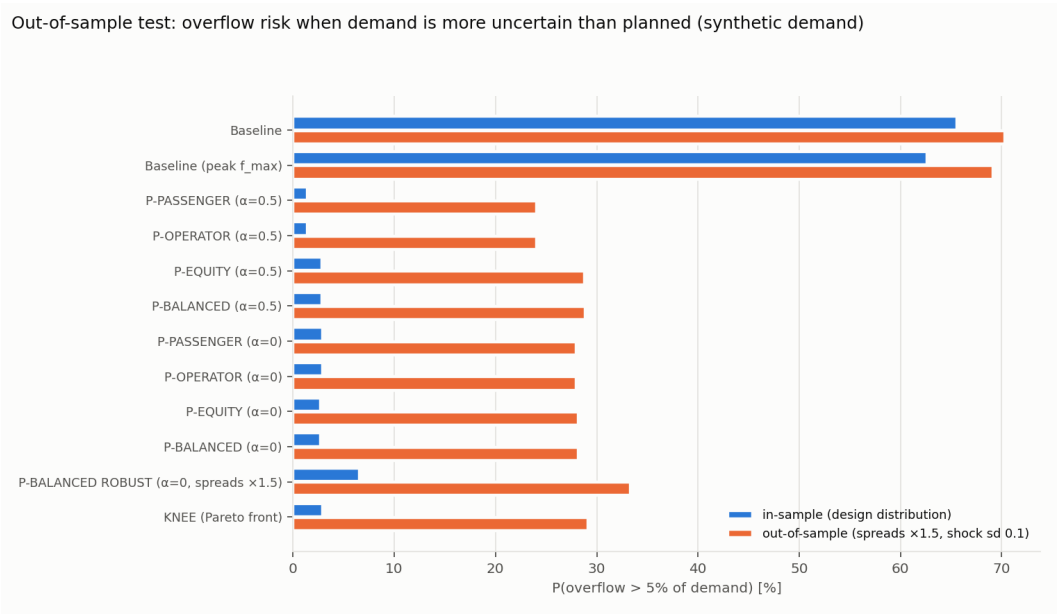
19.13 Fleet reduction



How to read: sensitivity_fleet.png

Bars: waiting improvement vs today when the usable fleet is 357, 339 (5% cut) or 321 (10% cut). Only 357 has a bar (+3.31%); the other two say "infeasible: minimum frequencies need more buses". Conclusion: 357 is close to the smallest fleet that can run today's minimum service.

19.14 Out-of-sample test

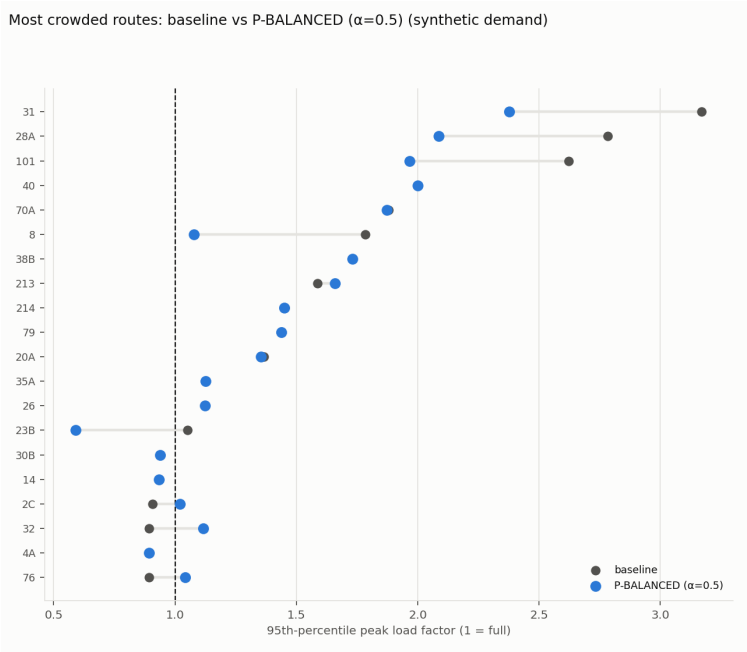


How to read: sensitivity_out_of_sample.png

- Pairs of bars per timetable: blue = probability of a bad day (overflow above 5%) on the demand it was planned for; orange = on wider, more uncertain demand it was not planned for.

- **Conclusion**: every timetable gets worse out of sample. Optimised plans go from about 1.5–3% to 24–29%; today's goes from 65.6% to 70.3%. The optimised plans are still much better, but the gain depends on the demand assumptions. The "robust" variant (yellow in other figures) is worse, not better (decision D47).

19.15 Most crowded routes



How to read: sim_route_p95_LF.png

- One row per route: the 20 most crowded today. **Across**: load factor on a bad day (95th percentile of the peak). 1 (dashed) = exactly full. Grey dot = today, blue dot = P-BALANCED; the line joins them.
- **Good**: blue to the left of grey. **Bad**: blue to the right.
- **Conclusion**: the worst routes (31, 28A, 101, 8, 23B) improve a lot, because they get more buses. Some routes (32, 2C, 76, 213) get a little more crowded, because buses were moved away from them. Several routes stay above 1: the fleet is not enough to fix every route.

Chapter 20

Results, limitations and what the project does not do

20.1 The headline, in numbers

Policy P-BALANCED (weighted goal programming, equal weights), $\alpha = 0.5$, relaxed $f_{\max}$, tested on 2000 simulated days (`outputs/simulation_findings.md`). **Synthetic demand, illustrative.**

Measure	Today	P-BALANCED	Change
expected wait per passenger	17.94 min	17.34 min	−3.3%
overflow (passengers not fitting)	2,249 /day	1,586 /day	−29.5%
vehicle-km	79,478 /day	77,045 /day	−3.1%
buses used	385	357	−7.3%
days with overflow > 5%	65.6%	2.9%	
minutes of headway above the cap	210	0	

The 2.9% is 57 of 2000 days, exactly 2.85%; `docs/DECISIONS.md` (D51) rounds the same number to 2.8%.

How it happens: buses move from routes with spare seats (32: 14 → 11 buses; 551, 206, 10, 20 each lose 2) to long, crowded, infrequent routes (31, 28A, 101: 80 → 60 min; 8: 75 → 45 min; 23B: 90 → 45 min) (`outputs/tables/gp_route_changes.csv`).

20.2 Five things to say about the results

1. The same city, with fewer buses on the road than today, can be timetabled to wait slightly less and crowd much less, *in this synthetic setting.*
2. The gain needs relaxed frequency limits. Under today's limits the model finds almost nothing to improve.
3. The exact method (AUGMECON) and the genetic algorithm (NSGA-II) agree: NSGA-II never beats an exact point.
4. The result depends most on speed and layover, which are assumptions. 18 km/h, 15% layover or 10% spares make the problem infeasible.
5. The result depends on the demand: on more uncertain demand, the bad-day probability of the optimised plan rises from 2.9% to 28.8% (today: 65.6% to 70.3%).

20.3 What the project does NOT do (known limitations)

1. **No real ridership.** All demand is synthetic; none of the numbers are CTU results.
2. **The demand anchor is unverified**: 143,000 daily riders and 684 buses are attributed to Tribune India reports but were not checked.
3. **One speed for the whole network** (22 km/h), no congestion, no time-of-day speeds.
4. **Route length meaning (one way) is inferred**, not confirmed by CTU.
5. **No passenger choice and no transfers**: demand does not react to better service.
6. **No depots, no drivers**: buses move freely between routes; no crew shifts or depot capacity.
7. **Only 07:00–19:00**, three periods; night and early-morning service is not optimised.
8. **Bunching is only in the waiting formula** (CV 0.3), not in capacity.
9. **Bus capacity 60 is an assumption** for every bus.
10. **Relaxed frequencies need CTU approval**, and terminal and stop capacity at the new headways is not checked.
11. **Equal weights in P-BALANCED are a judgement**; in the 3-objective space P-BALANCED is slightly dominated (distance 0.068), because it buys full equity with a little overflow.

20.4 Things that look strange but are correct

- **Fewer buses, better service.** Today's buses are not where the (synthetic) demand is; moving them helps more than adding them.
- **Model waiting -4.1%, simulated waiting -3.3%.** The model uses the most likely demand and $h/2$; the simulation uses random demand, the $1 + \mathrm{CV}^2$ factor and a per-passenger average.
- **G2 "zero overflow" is never met.** It is impossible with 357 buses; it works as "as little as possible".
- **The robust variant is worse.** Planning every route for 1.5 times wider demand cannot add capacity when the fleet is fixed (decision D47).

20.5 Where each number comes from

Topic	File
data audit, calibration	`outputs/calibration_report.md`
synthetic demand	`outputs/demand_report.md`
model size, baseline violations	`outputs/stage3_report.md`
goal programming	`outputs/gp_report.md`, `outputs/gp_results.csv`
Pareto front	`outputs/pareto_report.md`
simulation, sensitivity	`outputs/simulation_findings.md`
all headline numbers in one place	`outputs/report_facts.json`
every assumption	`docs/DECISIONS.md`

Check your understanding

1. Name two assumptions that could make the problem infeasible.

   Answer: Speed 18 km/h, layover 15%, spare ratio 10% (any two).

2. Why does the model need fewer buses but still improve waiting?

   Answer: It moves buses from routes with spare seats to crowded routes with long headways.

3. What should you say if asked "will CTU passengers wait 3.3% less?"

   Answer: Not necessarily: the demand is synthetic. The result shows the method can find such improvements; real ridership data is needed to claim them for CTU.

Glossary of terms and symbols

Symbols

Symbol	Meaning	Unit
r, t, k	route, period (AM / Midday / PM), menu option	–
h, h_k	headway: minutes between buses; menu value k	min
$f = 60/h$	frequency: departures per hour	dep/h
$f_{\min}, f_{\max}, f_{\text{mid}}$	lowest, highest, middle published frequency	dep/h
d_r	published route length	km
d_{cycle}	km per cycle ($2d_r$, or d_r for loops)	km
v	average speed (22)	km/h
ℓ	layover fraction (0.10)	–
τ	cycle time	h
x_{rt}, x_0	buses (decision), today's buses	buses
y_{rtk}	binary switch: headway k chosen	0/1
o_{rt}	overflow: demand above capacity	pax/h
B, B_{eff}, s	fleet (389), usable fleet (357), spare ratio (0.08)	buses, –
C	bus capacity (60)	pax/bus
H_t, H_{rt}	period length; service hours inside the period	h
M, L, U	most likely, low, high demand (TFN)	pax/h
α, D_α	cut level; planning demand $U - \alpha(U - M)$	–, pax/h
Z_1, Z_2, Z_3	waiting, vehicle-km, unserved passengers	pax · min/h, veh-km/day, pax/day
n_i, p_i	under- and over-achievement of goal i	goal's unit
κ_1, κ_3	target cuts for waiting (0.04) and km (0.03)	–
T_i	target of goal i	goal's unit
λ	largest normalised deviation (Chebyshev); also the demand scaling factor	–
ε, e_1, e_2	limits in the ε-constraint method	objective's unit
s_1, s_2, r_i	AUGMECON slacks and objective ranges	objective's unit
LF	load factor = demand / (Cf)	–
CV	coefficient of variation of headways (0.3)	–
EWT	excess wait time = $(h/2)\text{CV}^2$	min
$\lceil a \rceil$, $\lfloor a \rfloor$	round up, round down	–
$\sum$, $\in$	sum over; "is one of"	–

Terms

Term	Meaning (chapter)
active pair	a route-period in which the route runs; 191 in total (11)
α-cut	slice of a fuzzy number at plausibility α (13)
AUGMECON	augmented ε-constraint method for exact Pareto fronts (16)
baseline	today's published timetable evaluated by the same model (15)
binary variable	a variable that is 0 or 1 (8)
binding constraint	a constraint that holds with equality at the solution (4)
branch and bound	splitting an integer problem into sub-problems and pruning by bounds (9)
calibration	choosing speed, layover and length reading so the timetable fits the fleet (11)
Chebyshev GP	minimise the largest normalised goal miss (15)
common random numbers	testing every option on the same random draws (17)
constraint	a rule every solution must obey (3)
convex	a region with no dents (7)
corner point	where two borders of the feasible region meet (4, 6)
coverage $C(A, B)$	share of B matched or beaten by A (16)
crowding distance	how isolated a point is on its front; NSGA-II prefers isolated points (16)
decision variable	a number the solver chooses (1)
deviation variable	n or p, the distance from a goal's target (15)
dominance	better or equal everywhere, strictly better somewhere (14)
equity cap	60 min (peaks) / 90 min (midday) maximum headway goal (10, 15)
feasible / infeasible	obeys all constraints / no solution obeys them (4)
fuzzy number (TFN)	uncertain value described by (L, M, U) (13)
GD / IGD	average distance from heuristic to exact front / reverse (16)
genetic algorithm	evolution-style search: crossover, mutation, selection (16)
Gini coefficient	inequality measure, 0 = equal (17)
goal programming	optimising misses from targets instead of raw objectives (15)
headway	minutes between buses (1)
HiGHS	the solver used (7)
hypervolume	area/volume dominated by a front, up to a reference point (16)
ideal / nadir	best / worst value of each objective on the front (14)
incumbent	best integer solution found so far (9)
integer variable	whole-number variable (8)
knee	front point closest to the ideal (14)
layover	rest time at a terminal (11)
linear	only "number times variable" terms added together (2)
Little's law	buses needed $= f \times \tau$ (11)
load factor	demand divided by capacity (11)
lock	a constraint that keeps an earlier goal's achieved value (10, 15)
LP	linear program (6)
LP relaxation	the MILP without whole-number rules (9)
menu	the list of allowed headways (8, 10)
MILP	mixed-integer linear program (9)
MIP gap	distance between incumbent and bound (9)
Monte Carlo	many random scenarios to study spread of outcomes (17)

Term	Meaning (chapter)
normalisation	rescaling objectives to 0 (ideal) – 1 (nadir) (14)
NSGA-II	multi-objective genetic algorithm (16)
objective function	the number to minimise or maximise (5)
out-of-sample	testing on data the plan was not built for (17)
overflow	passengers above capacity (10, 11)
parameter	a fixed input number (1)
Pareto front	all non-dominated solutions (14)
payoff table	each objective optimised alone, all objectives recorded (14)
percentile	value below which a given share of results lies (17)
preemptive GP	goals optimised one by one in priority order (15)
PuLP	Python library used to write the model (7)
relaxed $f_{\max}$	allows headways up to two menu levels shorter than published (15)
repair	making a random genome satisfy the bus limits (16)
seed	starting value that makes random numbers repeatable (17)
sensitivity analysis	changing one assumption at a time (17)
simplex method	LP method that walks from corner to better corner (7)
slack	the unused room in a constraint (4)
solver	program that finds the optimum (7)
synthetic demand	invented by a stated formula, not measured (13)
tidy solve	final solve minimising buses with goals locked (15)
tornado chart	sensitivity bars sorted by size (17)
vehicle-km	kilometres driven by all buses (11)
weighted GP	one solve minimising weighted normalised misses (15)